Computación y Estructuras Discretas III

Andrés A. Aristizábal P.
aaaristizabal@icesi.edu.co

Departamento de Computación y Sistemas Inteligentes

2026-1

Agenda

1. **Grammars and Languages**
   - Topic presentation
   - Exercises

Agenda

1. **Grammars and Languages**
   - Topic presentation
   - Exercises

In CFG, what are useless variables?

In CFG, what are useless variables?

In a CFG, there can be two types of useless variables:

In CFG, what are useless variables?

In a CFG, there can be two types of useless variables:

1. Those that never appear in the course of a derivation

In CFG, what are useless variables?

In a CFG, there can be two types of useless variables:

1. Those that never appear in the course of a derivation
2. Those that cannot be converted into strings of terminals.

In CFG, what are useless variables?

In a CFG, there can be two types of useless variables:

1. Those that never appear in the course of a derivation
2. Those that cannot be converted into strings of terminals.

What is a reachable or accessible variable?

In CFG, what are useless variables?

In a CFG, there can be two types of useless variables:

1. Those that never appear in the course of a derivation
2. Those that cannot be converted into strings of terminals.

What is a reachable or accessible variable?

Definition

A variable A is reachable or accessible if there exist $u, v \in (V \cup \Sigma)^*$ such that $S \overset{*}{\Longrightarrow} uAv$. The initial variable S is reachable by definition.

What is a generating variable?

What is a generating variable?

Definition

A variable A is generating if there exists $w \in \Sigma^*$ such that $A \overset{*}{\Longrightarrow} w$. In particular, if $A \to \lambda$ is a production then A is generating.

What is a generating variable?

Definition

A variable A is generating if there exists $w \in \Sigma^*$ such that $A \overset{*}{\Longrightarrow} w$. In particular, if $A \to \lambda$ is a production then A is generating.

What is a useless variable?

What is a generating variable?

Definition

A variable A is generating if there exists $w \in \Sigma^*$ such that $A \overset{*}{\Longrightarrow} w$. In particular, if $A \to \lambda$ is a production then A is generating.

What is a useless variable?

Definition

A is a useless variable if it is non reachable or non generating.

What is a generating variable?

Definition

A variable A is generating if there exists $w \in \Sigma^*$ such that $A \overset{*}{\Longrightarrow} w$. In particular, if $A \to \lambda$ is a production then A is generating.

What is a useless variable?

Definition

A is a useless variable if it is non reachable or non generating.

How can a CFG be constructed without useless variables?

What is a generating variable?

Definition

A variable A is generating if there exists $w \in \Sigma^*$ such that $A \overset{*}{\Longrightarrow} w$. In particular, if $A \to \lambda$ is a production then A is generating.

What is a useless variable?

Definition

A is a useless variable if it is non reachable or non generating.

How can a CFG be constructed without useless variables?

There exist algorithms to detect all the useless variables of a CFG, allowing the construction of an equivalent grammar G' from a given grammar G, such that G' has no useless variables.

What is the algorithm to find generating variables in a CFG?

What is the algorithm to find generating variables in a CFG?

INITIALIZE:
  GEN $:= \{A \in V \mid \exists \text{ rule } A \to w,\ w \in \Sigma^*\}$
REPEAT:
  GEN $:=$ **GEN** $\cup \{A \in V \mid \exists \text{ rule } A \to w,\ w \in (\Sigma \cup \textbf{GEN})^*\}$
UNTIL:
  There are no more variables added to **GEN**

Example

Find the generating variables from this grammar:

Example

Find the generating variables from this grammar:

$$G : \begin{cases} S \to ACD \mid bBd \mid ab \\ A \to aB \mid aA \mid C \\ B \to aDS \mid aB \\ C \to aCS \mid CB \mid CC \\ D \to bD \mid ba \\ E \to AB \mid aDb \end{cases}$$

Example

Find the generating variables from this grammar:

$$G: \begin{cases} S \to ACD \mid bBd \mid ab \\ A \to aB \mid aA \mid C \\ B \to aDS \mid aB \\ C \to aCS \mid CB \mid CC \\ D \to bD \mid ba \\ E \to AB \mid aDb \end{cases}$$

Solution

$$\mathbf{GEN_1} = \{S, D\}.$$

Example

Find the generating variables from this grammar:

$$G: \begin{cases} S \to ACD \mid bBd \mid ab \\ A \to aB \mid aA \mid C \\ B \to aDS \mid aB \\ C \to aCS \mid CB \mid CC \\ D \to bD \mid ba \\ E \to AB \mid aDb \end{cases}$$

Solution

$$\mathbf{GEN_1} = \{S, D\}.$$
$$\mathbf{GEN_2} = \{S, D\} \cup \{B, E\} = \{S, D, B, E\}.$$

Example

Find the generating variables from this grammar:

$$G: \begin{cases} S \to ACD \mid bBd \mid ab \\ A \to aB \mid aA \mid C \\ B \to aDS \mid aB \\ C \to aCS \mid CB \mid CC \\ D \to bD \mid ba \\ E \to AB \mid aDb \end{cases}$$

Solution

$$\begin{aligned} \mathbf{GEN_1} &= \{S, D\}. \\ \mathbf{GEN_2} &= \{S, D\} \cup \{B, E\} = \{S, D, B, E\}. \\ \mathbf{GEN_3} &= \{S, D, B, E\} \cup \{A\} = \{S, D, B, E, A\}. \end{aligned}$$

Example

Find the generating variables from this grammar:

$$G: \begin{cases} S \to ACD \mid bBd \mid ab \\ A \to aB \mid aA \mid C \\ B \to aDS \mid aB \\ C \to aCS \mid CB \mid CC \\ D \to bD \mid ba \\ E \to AB \mid aDb \end{cases}$$

Solution

$$\begin{aligned} \mathbf{GEN_1} &= \{S, D\}. \\ \mathbf{GEN_2} &= \{S, D\} \cup \{B, E\} = \{S, D, B, E\}. \\ \mathbf{GEN_3} &= \{S, D, B, E\} \cup \{A\} = \{S, D, B, E, A\}. \\ \mathbf{GEN_4} &= \{S, D, B, E, A\} \cup \{\,\} = \{S, D, B, E, A\}. \end{aligned}$$

What is the algorithm to find reachable variables in a CFG?

Grammars and Languages

What is the algorithm to find reachable variables in a CFG?

INITIALIZE:
    REACH $:= \{S\}$
REPEAT:
    REACH $:=$ **REACH** $\cup \{A \in V \mid \exists$ producción $B \to uAv,\ B \in$ **REACH** y $u, v \in (V \cup \Sigma)^*\}$
UNTIL:
    There are no more variables added to **REACH**

Example

Find the reachable variables from this grammar:

Example

Find the reachable variables from this grammar:

$$G: \begin{cases} S \rightarrow aS \mid AaB \mid ACS \\ A \rightarrow aS \mid AaB \mid AC \\ B \rightarrow bB \mid DB \mid BB \\ C \rightarrow aDa \mid ABD \mid ab \\ D \rightarrow aD \mid DD \mid ab \\ E \rightarrow FF \mid aa \\ F \rightarrow aE \mid EF \end{cases}$$

Example

Find the reachable variables from this grammar:

$$G: \begin{cases} S \to aS \mid AaB \mid ACS \\ A \to aS \mid AaB \mid AC \\ B \to bB \mid DB \mid BB \\ C \to aDa \mid ABD \mid ab \\ D \to aD \mid DD \mid ab \\ E \to FF \mid aa \\ F \to aE \mid EF \end{cases}$$

Solution

$$\textbf{REACH}_1 = \{S\}.$$

Example

Find the reachable variables from this grammar:

$$G: \begin{cases} S \rightarrow aS \mid AaB \mid ACS \\ A \rightarrow aS \mid AaB \mid AC \\ B \rightarrow bB \mid DB \mid BB \\ C \rightarrow aDa \mid ABD \mid ab \\ D \rightarrow aD \mid DD \mid ab \\ E \rightarrow FF \mid aa \\ F \rightarrow aE \mid EF \end{cases}$$

Solution

$$\begin{aligned} \textbf{REACH}_1 &= \{S\}. \\ \textbf{REACH}_2 &= \{S\} \cup \{A, B, C\} = \{S, A, B, C\}. \end{aligned}$$

Example

Find the reachable variables from this grammar:

$$G: \begin{cases} S \to aS \mid AaB \mid ACS \\ A \to aS \mid AaB \mid AC \\ B \to bB \mid DB \mid BB \\ C \to aDa \mid ABD \mid ab \\ D \to aD \mid DD \mid ab \\ E \to FF \mid aa \\ F \to aE \mid EF \end{cases}$$

Solution

$$\begin{aligned} \textbf{REACH}_1 &= \{S\}. \\ \textbf{REACH}_2 &= \{S\} \cup \{A, B, C\} = \{S, A, B, C\}. \\ \textbf{REACH}_3 &= \{S, A, B, C\} \cup \{D\} = \{S, A, B, C, D\}. \end{aligned}$$

Example

Find the reachable variables from this grammar:

$$G: \begin{cases} S \rightarrow aS \mid AaB \mid ACS \\ A \rightarrow aS \mid AaB \mid AC \\ B \rightarrow bB \mid DB \mid BB \\ C \rightarrow aDa \mid ABD \mid ab \\ D \rightarrow aD \mid DD \mid ab \\ E \rightarrow FF \mid aa \\ F \rightarrow aE \mid EF \end{cases}$$

Solution

$$\begin{aligned} \textbf{REACH}_1 &= \{S\}. \\ \textbf{REACH}_2 &= \{S\} \cup \{A, B, C\} = \{S, A, B, C\}. \\ \textbf{REACH}_3 &= \{S, A, B, C\} \cup \{D\} = \{S, A, B, C, D\}. \\ \textbf{REACH}_4 &= \{S, A, B, C, D\} \cup \{\ \} = \{S, A, B, C, D\}. \end{aligned}$$

Example

Remove the useless variables from G.

Example

Remove the useless variables from G.

$$G: \begin{cases} S \to SBS \mid BC \mid Bb \\ A \to AA \mid aA \\ B \to aBCa \mid b \\ C \to aC \mid ACC \mid abb \\ D \to aAB \mid ab \\ E \to aS \mid bAA \\ F \to aDb \mid aF \end{cases}$$

Example

Remove the useless variables from G.

$$G: \begin{cases} S \to SBS \mid BC \mid Bb \\ A \to AA \mid aA \\ B \to aBCa \mid b \\ C \to aC \mid ACC \mid abb \\ D \to aAB \mid ab \\ E \to aS \mid bAA \\ F \to aDb \mid aF \end{cases}$$

Solution

1. *We execute the algorithm to find generating variables.*

Example

Remove the useless variables from G.

$$G : \begin{cases} S \to SBS \mid BC \mid Bb \\ A \to AA \mid aA \\ B \to aBCa \mid b \\ C \to aC \mid ACC \mid abb \\ D \to aAB \mid ab \\ E \to aS \mid bAA \\ F \to aDb \mid aF \end{cases}$$

Solution

1. *We execute the algorithm to find generating variables.*
2. *We eliminate non generating variables.*

Example

Remove the useless variables from G.

$$G : \begin{cases} S \to SBS \mid BC \mid Bb \\ A \to AA \mid aA \\ B \to aBCa \mid b \\ C \to aC \mid ACC \mid abb \\ D \to aAB \mid ab \\ E \to aS \mid bAA \\ F \to aDb \mid aF \end{cases}$$

Solution

1. *We execute the algorithm to find generating variables.*
2. *We eliminate non generating variables.*
3. *We execute the algorithm to find reachable variables.*

Example

Remove the useless variables from G.

$$G : \begin{cases} S \to SBS \mid BC \mid Bb \\ A \to AA \mid aA \\ B \to aBCa \mid b \\ C \to aC \mid ACC \mid abb \\ D \to aAB \mid ab \\ E \to aS \mid bAA \\ F \to aDb \mid aF \end{cases}$$

Solution

1. *We execute the algorithm to find generating variables.*
2. *We eliminate non generating variables.*
3. *We execute the algorithm to find reachable variables.*
4. *We eliminate non reachable variables.*

Solution

$$G : \begin{cases} S \rightarrow SBS \mid BC \mid Bb \\ A \rightarrow AA \mid aA \\ B \rightarrow aBCa \mid b \\ C \rightarrow aC \mid ACC \mid abb \\ D \rightarrow aAB \mid ab \\ E \rightarrow aS \mid bAA \\ F \rightarrow aDb \mid aF \end{cases}$$

Solution

$$G : \begin{cases} S \rightarrow SBS \mid BC \mid Bb \\ A \rightarrow AA \mid aA \\ B \rightarrow aBCa \mid b \\ C \rightarrow aC \mid ACC \mid abb \\ D \rightarrow aAB \mid ab \\ E \rightarrow aS \mid bAA \\ F \rightarrow aDb \mid aF \end{cases}$$

1. *We execute the first algorithm.*

Solution

$$G : \begin{cases} S \to SBS \mid BC \mid Bb \\ A \to AA \mid aA \\ B \to aBCa \mid b \\ C \to aC \mid ACC \mid abb \\ D \to aAB \mid ab \\ E \to aS \mid bAA \\ F \to aDb \mid aF \end{cases}$$

1. *We execute the first algorithm.*

$$\mathbf{GEN_1} = \{B, C, D\}.$$

Solution

$$G : \begin{cases} S \to SBS \mid BC \mid Bb \\ A \to AA \mid aA \\ B \to aBCa \mid b \\ C \to aC \mid ACC \mid abb \\ D \to aAB \mid ab \\ E \to aS \mid bAA \\ F \to aDb \mid aF \end{cases}$$

1. *We execute the first algorithm.*

$$\begin{aligned} \mathbf{GEN_1} &= \{B, C, D\}. \\ \mathbf{GEN_2} &= \{B, C, D\} \cup \{S, F\} = \{B, C, D, S, F\}. \end{aligned}$$

Solution

$$G: \begin{cases} S \to SBS \mid BC \mid Bb \\ A \to AA \mid aA \\ B \to aBCa \mid b \\ C \to aC \mid ACC \mid abb \\ D \to aAB \mid ab \\ E \to aS \mid bAA \\ F \to aDb \mid aF \end{cases}$$

1. *We execute the first algorithm.*

$$\begin{aligned} \mathbf{GEN_1} &= \{B, C, D\}. \\ \mathbf{GEN_2} &= \{B, C, D\} \cup \{S, F\} = \{B, C, D, S, F\}. \\ \mathbf{GEN_3} &= \{B, C, D, S, F\} \cup \{E\} = \{B, C, D, S, F, E\}. \end{aligned}$$

Solution

$$G: \begin{cases} S \to SBS \mid BC \mid Bb \\ A \to AA \mid aA \\ B \to aBCa \mid b \\ C \to aC \mid ACC \mid abb \\ D \to aAB \mid ab \\ E \to aS \mid bAA \\ F \to aDb \mid aF \end{cases}$$

1. *We execute the first algorithm.*

$$\begin{aligned} \mathbf{GEN_1} &= \{B, C, D\}. \\ \mathbf{GEN_2} &= \{B, C, D\} \cup \{S, F\} = \{B, C, D, S, F\}. \\ \mathbf{GEN_3} &= \{B, C, D, S, F\} \cup \{E\} = \{B, C, D, S, F, E\}. \\ \mathbf{GEN_4} &= \{B, C, D, S, F, E\} \cup \{\ \} = \{B, C, D, S, F, E\}. \end{aligned}$$

Solution

$$G : \begin{cases} S \to SBS \mid BC \mid Bb \\ A \to AA \mid aA \\ B \to aBCa \mid b \\ C \to aC \mid ACC \mid abb \\ D \to aAB \mid ab \\ E \to aS \mid bAA \\ F \to aDb \mid aF \end{cases}$$

1. *We execute the first algorithm.*

$$\begin{aligned} \mathbf{GEN_1} &= \{B, C, D\}. \\ \mathbf{GEN_2} &= \{B, C, D\} \cup \{S, F\} = \{B, C, D, S, F\}. \\ \mathbf{GEN_3} &= \{B, C, D, S, F\} \cup \{E\} = \{B, C, D, S, F, E\}. \\ \mathbf{GEN_4} &= \{B, C, D, S, F, E\} \cup \{\ \} = \{B, C, D, S, F, E\}. \end{aligned}$$

2. *The only non generating variable from G is A. Therefore, G is equivalent to the following grammar* G_1 :

Solution

$$G : \begin{cases} S \to SBS \mid BC \mid Bb \\ A \to AA \mid aA \\ B \to aBCa \mid b \\ C \to aC \mid ACC \mid abb \\ D \to aAB \mid ab \\ E \to aS \mid bAA \\ F \to aDb \mid aF \end{cases}$$

1. *We execute the first algorithm.*

$$\begin{aligned} \mathbf{GEN_1} &= \{B, C, D\}. \\ \mathbf{GEN_2} &= \{B, C, D\} \cup \{S, F\} = \{B, C, D, S, F\}. \\ \mathbf{GEN_3} &= \{B, C, D, S, F\} \cup \{E\} = \{B, C, D, S, F, E\}. \\ \mathbf{GEN_4} &= \{B, C, D, S, F, E\} \cup \{\ \} = \{B, C, D, S, F, E\}. \end{aligned}$$

2. *The only non generating variable from G is A. Therefore, G is equivalent to the following grammar* G_1 :

$$G_1 : \begin{cases} S \to SBS \mid BC \mid Bb \\ \\ \\ \end{cases}$$

Solution

$$G : \begin{cases} S \to SBS \mid BC \mid Bb \\ A \to AA \mid aA \\ B \to aBCa \mid b \\ C \to aC \mid ACC \mid abb \\ D \to aAB \mid ab \\ E \to aS \mid bAA \\ F \to aDb \mid aF \end{cases}$$

1. *We execute the first algorithm.*

$$\begin{aligned} \mathbf{GEN_1} &= \{B, C, D\}. \\ \mathbf{GEN_2} &= \{B, C, D\} \cup \{S, F\} = \{B, C, D, S, F\}. \\ \mathbf{GEN_3} &= \{B, C, D, S, F\} \cup \{E\} = \{B, C, D, S, F, E\}. \\ \mathbf{GEN_4} &= \{B, C, D, S, F, E\} \cup \{\ \} = \{B, C, D, S, F, E\}. \end{aligned}$$

2. *The only non generating variable from G is A. Therefore, G is equivalent to the following grammar* G_1 :

$$G_1 : \begin{cases} S \to SBS \mid BC \mid Bb \\ B \to aBCa \mid b \end{cases}$$

Solution

$$G : \begin{cases} S \to SBS \mid BC \mid Bb \\ A \to AA \mid aA \\ B \to aBCa \mid b \\ C \to aC \mid ACC \mid abb \\ D \to aAB \mid ab \\ E \to aS \mid bAA \\ F \to aDb \mid aF \end{cases}$$

1. *We execute the first algorithm.*

$$\begin{aligned} \mathbf{GEN_1} &= \{B, C, D\}. \\ \mathbf{GEN_2} &= \{B, C, D\} \cup \{S, F\} = \{B, C, D, S, F\}. \\ \mathbf{GEN_3} &= \{B, C, D, S, F\} \cup \{E\} = \{B, C, D, S, F, E\}. \\ \mathbf{GEN_4} &= \{B, C, D, S, F, E\} \cup \{\ \} = \{B, C, D, S, F, E\}. \end{aligned}$$

2. *The only non generating variable from G is A. Therefore, G is equivalent to the following grammar* G_1 :

$$G_1 : \begin{cases} S \to SBS \mid BC \mid Bb \\ B \to aBCa \mid b \\ C \to aC \mid abb \end{cases}$$

Solution

$$G : \begin{cases} S \to SBS \mid BC \mid Bb \\ A \to AA \mid aA \\ B \to aBCa \mid b \\ C \to aC \mid ACC \mid abb \\ D \to aAB \mid ab \\ E \to aS \mid bAA \\ F \to aDb \mid aF \end{cases}$$

1. *We execute the first algorithm.*

$$\begin{aligned} \mathbf{GEN_1} &= \{B, C, D\}. \\ \mathbf{GEN_2} &= \{B, C, D\} \cup \{S, F\} = \{B, C, D, S, F\}. \\ \mathbf{GEN_3} &= \{B, C, D, S, F\} \cup \{E\} = \{B, C, D, S, F, E\}. \\ \mathbf{GEN_4} &= \{B, C, D, S, F, E\} \cup \{\ \} = \{B, C, D, S, F, E\}. \end{aligned}$$

2. *The only non generating variable from G is A. Therefore, G is equivalent to the following grammar* G_1 :

$$G_1 : \begin{cases} S \to SBS \mid BC \mid Bb \\ B \to aBCa \mid b \\ C \to aC \mid abb \\ D \to ab \end{cases}$$

Solution

$$G : \begin{cases} S \to SBS \mid BC \mid Bb \\ A \to AA \mid aA \\ B \to aBCa \mid b \\ C \to aC \mid ACC \mid abb \\ D \to aAB \mid ab \\ E \to aS \mid bAA \\ F \to aDb \mid aF \end{cases}$$

1. *We execute the first algorithm.*

$$\begin{aligned} \mathbf{GEN_1} &= \{B, C, D\}. \\ \mathbf{GEN_2} &= \{B, C, D\} \cup \{S, F\} = \{B, C, D, S, F\}. \\ \mathbf{GEN_3} &= \{B, C, D, S, F\} \cup \{E\} = \{B, C, D, S, F, E\}. \\ \mathbf{GEN_4} &= \{B, C, D, S, F, E\} \cup \{\ \} = \{B, C, D, S, F, E\}. \end{aligned}$$

2. *The only non generating variable from G is A. Therefore, G is equivalent to the following grammar* G_1 :

$$G_1 : \begin{cases} S \to SBS \mid BC \mid Bb \\ B \to aBCa \mid b \\ C \to aC \mid abb \\ D \to ab \\ E \to aS \end{cases}$$

Solution

$$G : \begin{cases} S \to SBS \mid BC \mid Bb \\ A \to AA \mid aA \\ B \to aBCa \mid b \\ C \to aC \mid ACC \mid abb \\ D \to aAB \mid ab \\ E \to aS \mid bAA \\ F \to aDb \mid aF \end{cases}$$

1. *We execute the first algorithm.*

$$\begin{aligned} \mathbf{GEN_1} &= \{B, C, D\}. \\ \mathbf{GEN_2} &= \{B, C, D\} \cup \{S, F\} = \{B, C, D, S, F\}. \\ \mathbf{GEN_3} &= \{B, C, D, S, F\} \cup \{E\} = \{B, C, D, S, F, E\}. \\ \mathbf{GEN_4} &= \{B, C, D, S, F, E\} \cup \{\ \} = \{B, C, D, S, F, E\}. \end{aligned}$$

2. *The only non generating variable from G is A. Therefore, G is equivalent to the following grammar* G_1 :

$$G_1 : \begin{cases} S \to SBS \mid BC \mid Bb \\ B \to aBCa \mid b \\ C \to aC \mid abb \\ D \to ab \\ E \to aS \\ F \to aDb \mid aF \end{cases}$$

Solution

$$G_1 : \begin{cases} S \rightarrow SBS \mid BC \mid Bb \\ B \rightarrow aBCa \mid b \\ C \rightarrow aC \mid abb \\ D \rightarrow ab \\ E \rightarrow aS \\ F \rightarrow aDb \mid aF \end{cases}$$

1. *We execute the second algorithm.*

Solution

$$G_1 : \begin{cases} S \to SBS \mid BC \mid Bb \\ B \to aBCa \mid b \\ C \to aC \mid abb \\ D \to ab \\ E \to aS \\ F \to aDb \mid aF \end{cases}$$

1. *We execute the second algorithm.*

$$\mathbf{REACH_1} = \{S\}.$$

Solution

$$G_1 : \begin{cases} S \to SBS \mid BC \mid Bb \\ B \to aBCa \mid b \\ C \to aC \mid abb \\ D \to ab \\ E \to aS \\ F \to aDb \mid aF \end{cases}$$

1. *We execute the second algorithm.*

$$\begin{aligned} \textbf{REACH}_1 &= \{S\}. \\ \textbf{REACH}_2 &= \{S\} \cup \{B, C\} = \{S, B, C\}. \end{aligned}$$

Solution

$$G_1 : \begin{cases} S \to SBS \mid BC \mid Bb \\ B \to aBCa \mid b \\ C \to aC \mid abb \\ D \to ab \\ E \to aS \\ F \to aDb \mid aF \end{cases}$$

1. *We execute the second algorithm.*

$$\begin{aligned} \textbf{REACH}_1 &= \{S\}. \\ \textbf{REACH}_2 &= \{S\} \cup \{B, C\} = \{S, B, C\}. \\ \textbf{REACH}_3 &= \{S, B, C\} \cup \{\ \} = \{S, B, C\}. \end{aligned}$$

Solution

$$G_1 : \begin{cases} S \to SBS \mid BC \mid Bb \\ B \to aBCa \mid b \\ C \to aC \mid abb \\ D \to ab \\ E \to aS \\ F \to aDb \mid aF \end{cases}$$

1. *We execute the second algorithm.*

$$\begin{aligned} \mathbf{REACH_1} &= \{S\}. \\ \mathbf{REACH_2} &= \{S\} \cup \{B, C\} = \{S, B, C\}. \\ \mathbf{REACH_3} &= \{S, B, C\} \cup \{\ \} = \{S, B, C\}. \end{aligned}$$

2. *Variables D, E, F are non reachable. Therefore, G is equivalent to the following grammar G_2, that does not have useless variables.*

Solution

$$G_1 : \begin{cases} S \to SBS \mid BC \mid Bb \\ B \to aBCa \mid b \\ C \to aC \mid abb \\ D \to ab \\ E \to aS \\ F \to aDb \mid aF \end{cases}$$

1. *We execute the second algorithm.*

$$\begin{aligned} \mathbf{REACH_1} &= \{S\}. \\ \mathbf{REACH_2} &= \{S\} \cup \{B, C\} = \{S, B, C\}. \\ \mathbf{REACH_3} &= \{S, B, C\} \cup \{\ \} = \{S, B, C\}. \end{aligned}$$

2. *Variables D, E, F are non reachable. Therefore, G is equivalent to the following grammar G_2, that does not have useless variables.*

$$G_2 : \begin{cases} S \to SBS \mid BC \mid Bb \\ \\ \end{cases}$$

Solution

$$G_1 : \begin{cases} S \to SBS \mid BC \mid Bb \\ B \to aBCa \mid b \\ C \to aC \mid abb \\ D \to ab \\ E \to aS \\ F \to aDb \mid aF \end{cases}$$

1. *We execute the second algorithm.*

$$\begin{aligned} \mathbf{REACH_1} &= \{S\}. \\ \mathbf{REACH_2} &= \{S\} \cup \{B, C\} = \{S, B, C\}. \\ \mathbf{REACH_3} &= \{S, B, C\} \cup \{\ \} = \{S, B, C\}. \end{aligned}$$

2. *Variables D, E, F are non reachable. Therefore, G is equivalent to the following grammar G_2, that does not have useless variables.*

$$G_2 : \begin{cases} S \to SBS \mid BC \mid Bb \\ B \to aBCa \mid b \end{cases}$$

Solution

$$G_1 : \begin{cases} S \to SBS \mid BC \mid Bb \\ B \to aBCa \mid b \\ C \to aC \mid abb \\ D \to ab \\ E \to aS \\ F \to aDb \mid aF \end{cases}$$

1. *We execute the second algorithm.*

$$\begin{aligned} \mathbf{REACH_1} &= \{S\}. \\ \mathbf{REACH_2} &= \{S\} \cup \{B, C\} = \{S, B, C\}. \\ \mathbf{REACH_3} &= \{S, B, C\} \cup \{\ \} = \{S, B, C\}. \end{aligned}$$

2. *Variables D, E, F are non reachable. Therefore, G is equivalent to the following grammar G_2, that does not have useless variables.*

$$G_2 : \begin{cases} S \to SBS \mid BC \mid Bb \\ B \to aBCa \mid b \\ C \to aC \mid abb \end{cases}$$

Grammars and Languages

What is a λ rule?

What is a λ rule?

Definition

A rule of the form $A \to \lambda$ is called a λ rule.

What is a λ rule?

Definition

A rule of the form $A \to \lambda$ is called a λ rule.

What is a nullable variable?

Grammars and Languages

What is a λ rule?

Definition

A rule of the form $A \to \lambda$ is called a λ rule.

What is a nullable variable?

Definition

A variable A is nullable if $A \overset{*}{\Longrightarrow} \lambda$.

Grammars and Languages

What is a λ rule?

Definition

A rule of the form $A \to \lambda$ is called a λ rule.

What is a nullable variable?

Definition

A variable A is nullable if $A \overset{*}{\Longrightarrow} \lambda$.

What is the algorithm to find nullable variables in a CFG?

What is a λ rule?

Definition

A rule of the form $A \to \lambda$ is called a λ rule.

What is a nullable variable?

Definition

A variable A is nullable if $A \overset{*}{\Longrightarrow} \lambda$.

What is the algorithm to find nullable variables in a CFG?

INITIALIZE:
    NULL $:= \{A \in B \mid A \to \lambda \text{ is a rule}\}$
REPEAT:
    NULL $:=$ **NULL** $\cup \{A \in V \mid \exists \text{ rule } A \to w,\ w \in (\textbf{NULL})^*\}$
UNTIL:
    There are no more variables added to **NULL**

Grammars and Languages

Can you build a CFG without λ productions?

Theorem

Given a CFG G, can we construct an equivalent CFG G' to G without λ productions, except (possibly) $S \to \lambda$?

Grammars and Languages

Can you build a CFG without λ productions?

Theorem

Given a CFG G, can we construct an equivalent CFG G' to G without λ productions, except (possibly) $S \to \lambda$?

Example

Remove the λ rules from the following grammar G.

Grammars and Languages

Can you build a CFG without λ productions?

Theorem

Given a CFG G, can we construct an equivalent CFG G' to G without λ productions, except (possibly) $S \to \lambda$?

Example

Remove the λ rules from the following grammar G.

$$G : \begin{cases} S \to AB \mid ACA \mid ab \\ A \to aAa \mid B \mid CD \\ B \to bB \mid bA \\ C \to cC \mid \lambda \\ D \to aDc \mid CC \mid ABb \end{cases}$$

Can you build a CFG without λ productions?

Theorem

Given a CFG G, can we construct an equivalent CFG G' to G without λ productions, except (possibly) $S \to \lambda$?

Example

Remove the λ rules from the following grammar G.

$$G : \begin{cases} S \to AB \mid ACA \mid ab \\ A \to aAa \mid B \mid CD \\ B \to bB \mid bA \\ C \to cC \mid \lambda \\ D \to aDc \mid CC \mid ABb \end{cases}$$

Solution

1. *We execute the algorithm to find the nullable variables.*

Can you build a CFG without λ productions?

Theorem

Given a CFG G, can we construct an equivalent CFG G' to G without λ productions, except (possibly) $S \to \lambda$?

Example

Remove the λ rules from the following grammar G.

$$G : \begin{cases} S \to AB \mid ACA \mid ab \\ A \to aAa \mid B \mid CD \\ B \to bB \mid bA \\ C \to cC \mid \lambda \\ D \to aDc \mid CC \mid ABb \end{cases}$$

Solution

1. *We execute the algorithm to find the nullable variables.*
2. *We remove λ productions except for $S \to \lambda$, adding new productions that simulate the effect of the eliminated λ productions.*

Can you build a CFG without λ productions?

Theorem

Given a CFG G, can we construct an equivalent CFG G' to G without λ productions, except (possibly) $S \to \lambda$?

Example

Remove the λ rules from the following grammar G.

$$G : \begin{cases} S \to AB \mid ACA \mid ab \\ A \to aAa \mid B \mid CD \\ B \to bB \mid bA \\ C \to cC \mid \lambda \\ D \to aDc \mid CC \mid ABb \end{cases}$$

Solution

1. *We execute the algorithm to find the nullable variables.*
2. *We remove λ productions except for $S \to \lambda$, adding new productions that simulate the effect of the eliminated λ productions. That is, for each production $A \to u$ in G, we add productions of the form $A \to v$ obtained by removing from the string u one, two, or more nullable variables present, in all possible ways.*

Solution

$$G : \begin{cases} S \rightarrow AB \mid ACA \mid ab \\ A \rightarrow aAa \mid B \mid CD \\ B \rightarrow bB \mid bA \\ C \rightarrow cC \mid \lambda \\ D \rightarrow aDc \mid CC \mid ABb \end{cases}$$

Solution

$$G : \begin{cases} S \rightarrow AB \mid ACA \mid ab \\ A \rightarrow aAa \mid B \mid CD \\ B \rightarrow bB \mid bA \\ C \rightarrow cC \mid \lambda \\ D \rightarrow aDc \mid CC \mid ABb \end{cases}$$

1. *We execute the algorithm to find the nullable variables.*

Solution

$$G: \begin{cases} S \rightarrow AB \mid ACA \mid ab \\ A \rightarrow aAa \mid B \mid CD \\ B \rightarrow bB \mid bA \\ C \rightarrow cC \mid \lambda \\ D \rightarrow aDc \mid CC \mid ABb \end{cases}$$

1. *We execute the algorithm to find the nullable variables.*

$$\mathbf{NULL_1} = \{C\}.$$

Solution

$$G: \begin{cases} S \rightarrow AB \mid ACA \mid ab \\ A \rightarrow aAa \mid B \mid CD \\ B \rightarrow bB \mid bA \\ C \rightarrow cC \mid \lambda \\ D \rightarrow aDc \mid CC \mid ABb \end{cases}$$

1. *We execute the algorithm to find the nullable variables.*

$$\mathbf{NULL_1} = \{C\}.$$
$$\mathbf{NULL_2} = \{C\} \cup \{D\} = \{C, D\}.$$

Solution

$$G: \begin{cases} S \rightarrow AB \mid ACA \mid ab \\ A \rightarrow aAa \mid B \mid CD \\ B \rightarrow bB \mid bA \\ C \rightarrow cC \mid \lambda \\ D \rightarrow aDc \mid CC \mid ABb \end{cases}$$

1. *We execute the algorithm to find the nullable variables.*

$$\begin{aligned} \mathbf{NULL_1} &= \{C\}. \\ \mathbf{NULL_2} &= \{C\} \cup \{D\} = \{C, D\}. \\ \mathbf{NULL_3} &= \{C, D\} \cup \{A\} = \{C, D, A\}. \end{aligned}$$

Solution

$$G: \begin{cases} S \rightarrow AB \mid ACA \mid ab \\ A \rightarrow aAa \mid B \mid CD \\ B \rightarrow bB \mid bA \\ C \rightarrow cC \mid \lambda \\ D \rightarrow aDc \mid CC \mid ABb \end{cases}$$

1. *We execute the algorithm to find the nullable variables.*

$$\begin{aligned} \mathbf{NULL_1} &= \{C\}. \\ \mathbf{NULL_2} &= \{C\} \cup \{D\} = \{C, D\}. \\ \mathbf{NULL_3} &= \{C, D\} \cup \{A\} = \{C, D, A\}. \\ \mathbf{NULL_4} &= \{C, D, A\} \cup \{S\} = \{C, D, A, S\}. \end{aligned}$$

Solution

$$G: \begin{cases} S \rightarrow AB \mid ACA \mid ab \\ A \rightarrow aAa \mid B \mid CD \\ B \rightarrow bB \mid bA \\ C \rightarrow cC \mid \lambda \\ D \rightarrow aDc \mid CC \mid ABb \end{cases}$$

1. *We execute the algorithm to find the nullable variables.*

$$\begin{aligned} \mathbf{NULL_1} &= \{C\}. \\ \mathbf{NULL_2} &= \{C\} \cup \{D\} = \{C, D\}. \\ \mathbf{NULL_3} &= \{C, D\} \cup \{A\} = \{C, D, A\}. \\ \mathbf{NULL_4} &= \{C, D, A\} \cup \{S\} = \{C, D, A, S\}. \\ \mathbf{NULL_5} &= \{C, D, A, S\} \cup \{\ \} = \{C, D, A, S\}. \end{aligned}$$

Solution

$$G: \begin{cases} S \rightarrow AB \mid ACA \mid ab \\ A \rightarrow aAa \mid B \mid CD \\ B \rightarrow bB \mid bA \\ C \rightarrow cC \mid \lambda \\ D \rightarrow aDc \mid CC \mid ABb \end{cases}$$

1. *We execute the algorithm to find the nullable variables.*

$$\begin{aligned} \mathbf{NULL_1} &= \{C\}. \\ \mathbf{NULL_2} &= \{C\} \cup \{D\} = \{C, D\}. \\ \mathbf{NULL_3} &= \{C, D\} \cup \{A\} = \{C, D, A\}. \\ \mathbf{NULL_4} &= \{C, D, A\} \cup \{S\} = \{C, D, A, S\}. \\ \mathbf{NULL_5} &= \{C, D, A, S\} \cup \{\ \} = \{C, D, A, S\}. \end{aligned}$$

2. *By removing the λ productions from G (the only one being $C \rightarrow \lambda$) and adding the new productions that simulate the effect of the λ productions, we obtain the following grammar equivalent to G:*

Solution

$$G: \begin{cases} S \rightarrow AB \mid ACA \mid ab \\ A \rightarrow aAa \mid B \mid CD \\ B \rightarrow bB \mid bA \\ C \rightarrow cC \mid \lambda \\ D \rightarrow aDc \mid CC \mid ABb \end{cases}$$

1. *We execute the algorithm to find the nullable variables.*

$$\begin{aligned} \mathbf{NULL_1} &= \{C\}. \\ \mathbf{NULL_2} &= \{C\} \cup \{D\} = \{C, D\}. \\ \mathbf{NULL_3} &= \{C, D\} \cup \{A\} = \{C, D, A\}. \\ \mathbf{NULL_4} &= \{C, D, A\} \cup \{S\} = \{C, D, A, S\}. \\ \mathbf{NULL_5} &= \{C, D, A, S\} \cup \{\ \} = \{C, D, A, S\}. \end{aligned}$$

2. *By removing the λ productions from G (the only one being $C \rightarrow \lambda$) and adding the new productions that simulate the effect of the λ productions, we obtain the following grammar equivalent to G:*

$$G_1: \begin{cases} S \rightarrow AB \mid ACA \mid ab \mid B \mid CA \mid AA \mid AC \mid A \mid C \mid \lambda \\ \\ \\ \end{cases}$$

Solution

$$G: \begin{cases} S \rightarrow AB \mid ACA \mid ab \\ A \rightarrow aAa \mid B \mid CD \\ B \rightarrow bB \mid bA \\ C \rightarrow cC \mid \lambda \\ D \rightarrow aDc \mid CC \mid ABb \end{cases}$$

1. *We execute the algorithm to find the nullable variables.*

$$\begin{aligned} \mathbf{NULL_1} &= \{C\}. \\ \mathbf{NULL_2} &= \{C\} \cup \{D\} = \{C, D\}. \\ \mathbf{NULL_3} &= \{C, D\} \cup \{A\} = \{C, D, A\}. \\ \mathbf{NULL_4} &= \{C, D, A\} \cup \{S\} = \{C, D, A, S\}. \\ \mathbf{NULL_5} &= \{C, D, A, S\} \cup \{\ \} = \{C, D, A, S\}. \end{aligned}$$

2. *By removing the λ productions from G (the only one being $C \rightarrow \lambda$) and adding the new productions that simulate the effect of the λ productions, we obtain the following grammar equivalent to G:*

$$G_1: \begin{cases} S \rightarrow AB \mid ACA \mid ab \mid B \mid CA \mid AA \mid AC \mid A \mid C \mid \lambda \\ A \rightarrow aAa \mid B \mid CD \mid aa \mid C \mid D \end{cases}$$

Solution

$$G: \begin{cases} S \rightarrow AB \mid ACA \mid ab \\ A \rightarrow aAa \mid B \mid CD \\ B \rightarrow bB \mid bA \\ C \rightarrow cC \mid \lambda \\ D \rightarrow aDc \mid CC \mid ABb \end{cases}$$

1. *We execute the algorithm to find the nullable variables.*

$$\begin{aligned} \mathbf{NULL_1} &= \{C\}. \\ \mathbf{NULL_2} &= \{C\} \cup \{D\} = \{C, D\}. \\ \mathbf{NULL_3} &= \{C, D\} \cup \{A\} = \{C, D, A\}. \\ \mathbf{NULL_4} &= \{C, D, A\} \cup \{S\} = \{C, D, A, S\}. \\ \mathbf{NULL_5} &= \{C, D, A, S\} \cup \{\ \} = \{C, D, A, S\}. \end{aligned}$$

2. *By removing the λ productions from G (the only one being $C \rightarrow \lambda$) and adding the new productions that simulate the effect of the λ productions, we obtain the following grammar equivalent to G:*

$$G_1: \begin{cases} S \rightarrow AB \mid ACA \mid ab \mid B \mid CA \mid AA \mid AC \mid A \mid C \mid \lambda \\ A \rightarrow aAa \mid B \mid CD \mid aa \mid C \mid D \\ B \rightarrow bB \mid bA \mid b \end{cases}$$

Solution

$$G: \begin{cases} S \rightarrow AB \mid ACA \mid ab \\ A \rightarrow aAa \mid B \mid CD \\ B \rightarrow bB \mid bA \\ C \rightarrow cC \mid \lambda \\ D \rightarrow aDc \mid CC \mid ABb \end{cases}$$

1. *We execute the algorithm to find the nullable variables.*

$$\begin{aligned} \mathbf{NULL_1} &= \{C\}. \\ \mathbf{NULL_2} &= \{C\} \cup \{D\} = \{C, D\}. \\ \mathbf{NULL_3} &= \{C, D\} \cup \{A\} = \{C, D, A\}. \\ \mathbf{NULL_4} &= \{C, D, A\} \cup \{S\} = \{C, D, A, S\}. \\ \mathbf{NULL_5} &= \{C, D, A, S\} \cup \{\ \} = \{C, D, A, S\}. \end{aligned}$$

2. *By removing the λ productions from G (the only one being $C \rightarrow \lambda$) and adding the new productions that simulate the effect of the λ productions, we obtain the following grammar equivalent to G:*

$$G_1: \begin{cases} S \rightarrow AB \mid ACA \mid ab \mid B \mid CA \mid AA \mid AC \mid A \mid C \mid \lambda \\ A \rightarrow aAa \mid B \mid CD \mid aa \mid C \mid D \\ B \rightarrow bB \mid bA \mid b \\ C \rightarrow cC \mid c \end{cases}$$

Solution

$$G: \begin{cases} S \rightarrow AB \mid ACA \mid ab \\ A \rightarrow aAa \mid B \mid CD \\ B \rightarrow bB \mid bA \\ C \rightarrow cC \mid \lambda \\ D \rightarrow aDc \mid CC \mid ABb \end{cases}$$

1. *We execute the algorithm to find the nullable variables.*

$$\begin{aligned} \mathbf{NULL_1} &= \{C\}. \\ \mathbf{NULL_2} &= \{C\} \cup \{D\} = \{C, D\}. \\ \mathbf{NULL_3} &= \{C, D\} \cup \{A\} = \{C, D, A\}. \\ \mathbf{NULL_4} &= \{C, D, A\} \cup \{S\} = \{C, D, A, S\}. \\ \mathbf{NULL_5} &= \{C, D, A, S\} \cup \{\ \} = \{C, D, A, S\}. \end{aligned}$$

2. *By removing the λ productions from G (the only one being $C \rightarrow \lambda$) and adding the new productions that simulate the effect of the λ productions, we obtain the following grammar equivalent to G:*

$$G_1: \begin{cases} S \rightarrow AB \mid ACA \mid ab \mid B \mid CA \mid AA \mid AC \mid A \mid C \mid \lambda \\ A \rightarrow aAa \mid B \mid CD \mid aa \mid C \mid D \\ B \rightarrow bB \mid bA \mid b \\ C \rightarrow cC \mid c \\ D \rightarrow aDc \mid CC \mid ABb \mid ac \mid C \mid Bb \end{cases}$$

What is a unit production?

What is a unit production?

Definition

A production of the form $A \to B$ where A and B are variables is called a unit production.

What is a unit production?

Definition

A production of the form $A \to B$ where A and B are variables is called a unit production.

What is the unit set of a variable?

What is a unit production?

Definition

A production of the form $A \to B$ where A and B are variables is called a unit production.

What is the unit set of a variable?

Definition

The unit set of a variable A is defined as follows::

What is a unit production?

Definition

A production of the form $A \to B$ where A and B are variables is called a unit production.

What is the unit set of a variable?

Definition

The unit set of a variable A is defined as follows::

$$\textbf{\textit{UNIT}}(A) := \{X \in V \mid \exists \text{ a derivation } A \overset{*}{\Longrightarrow} X \text{ that uses only unit productions}\}.$$

What is a unit production?

Definition

A production of the form $A \to B$ where A and B are variables is called a unit production.

What is the unit set of a variable?

Definition

The unit set of a variable A is defined as follows::

$$\textbf{\textit{UNIT}}(A) := \{X \in V \mid \exists \text{ a derivation } A \overset{*}{\Longrightarrow} X \text{ that uses only unit productions}\}.$$

*By definition, $A \in$ **UNIT**(A).*

What is the algorithm to find the unit variables of a CFG?

What is the algorithm to find the unit variables of a CFG?

INITIALIZE:
    UNIT(A) $:= \{A\}$
REPEAT:
    UNIT(A) $:=$ **UNIT**$(A) \cup \{X \in V \mid \exists$ a production $Y \to X$, with $Y \in$ **UNIT**$(A)\}$
UNTIL:
    There are no more variables added to **UNIT**(A)

What is the algorithm to find the unit variables of a CFG?

INITIALIZE:
    UNIT(A) $:= \{A\}$
REPEAT:
    UNIT(A) $:=$ **UNIT**$(A) \cup \{X \in V \mid \exists$ a production $Y \to X$, with $Y \in$ **UNIT**$(A)\}$
UNTIL:
    There are no more variables added to **UNIT**(A)

Can a CFG be constructed without unit productions?

What is the algorithm to find the unit variables of a CFG?

INITIALIZE:
    UNIT(A) $:= \{A\}$
REPEAT:
    UNIT(A) $:=$ **UNIT**$(A) \cup \{X \in V \mid \exists$ a production $Y \to X$, with $Y \in$ **UNIT**$(A)\}$
UNTIL:
    There are no more variables added to **UNIT**(A)

Can a CFG be constructed without unit productions?

Theorem

Given a CFG G, it is possible to construct an equivalent to G CFG G' without unit productions.

Example

Remove the unit productions from grammar G.

Example

Remove the unit productions from grammar G.

$$G: \begin{cases} S \to AS \mid AA \mid BA \mid \lambda \\ A \to aA \mid a \\ B \to bB \mid bC \mid C \\ C \to aA \mid bA \mid B \mid ab \end{cases}$$

Example

Remove the unit productions from grammar G.

$$G: \begin{cases} S \to AS \mid AA \mid BA \mid \lambda \\ A \to aA \mid a \\ B \to bB \mid bC \mid C \\ C \to aA \mid bA \mid B \mid ab \end{cases}$$

Solution

1. *We execute the algorithm to find the unit variables.*

Example

Remove the unit productions from grammar G.

$$G : \begin{cases} S \to AS \mid AA \mid BA \mid \lambda \\ A \to aA \mid a \\ B \to bB \mid bC \mid C \\ C \to aA \mid bA \mid B \mid ab \end{cases}$$

Solution

1. *We execute the algorithm to find the unit variables.*
2. *We remove the unit productions, adding for each variable A of G the (non-unit) productions of the variables contained in the unit set* ***UNIT***(A).

Solution

$$G : \begin{cases} S \rightarrow AS \mid AA \mid BA \mid \lambda \\ A \rightarrow aA \mid a \\ B \rightarrow bB \mid bC \mid C \\ C \rightarrow aA \mid bA \mid B \mid ab \end{cases}$$

Solution

$$G : \begin{cases} S \rightarrow AS \mid AA \mid BA \mid \lambda \\ A \rightarrow aA \mid a \\ B \rightarrow bB \mid bC \mid C \\ C \rightarrow aA \mid bA \mid B \mid ab \end{cases}$$

1. *We execute the algorithm to find the unit productions.*

Solution

$$G : \begin{cases} S \to AS \mid AA \mid BA \mid \lambda \\ A \to aA \mid a \\ B \to bB \mid bC \mid C \\ C \to aA \mid bA \mid B \mid ab \end{cases}$$

1. *We execute the algorithm to find the unit productions.*

$$\mathbf{UNIT_1}(S) = \{S\}.$$

Solution

$$G : \begin{cases} S \to AS \mid AA \mid BA \mid \lambda \\ A \to aA \mid a \\ B \to bB \mid bC \mid C \\ C \to aA \mid bA \mid B \mid ab \end{cases}$$

1. *We execute the algorithm to find the unit productions.*

$$\mathbf{UNIT_1}(S) = \{S\}.$$
$$\mathbf{UNIT_2}(S) = \{S\} \cup \{\ \} = \{S\}.$$

Solution

$$G : \begin{cases} S \to AS \mid AA \mid BA \mid \lambda \\ A \to aA \mid a \\ B \to bB \mid bC \mid C \\ C \to aA \mid bA \mid B \mid ab \end{cases}$$

1. *We execute the algorithm to find the unit productions.*

$$\begin{aligned} &\mathbf{UNIT_1}(S) = \{S\}. \\ &\mathbf{UNIT_2}(S) = \{S\} \cup \{\ \} = \{S\}. \\ &\mathbf{UNIT_1}(A) = \{A\}. \end{aligned}$$

Solution

$$G : \begin{cases} S \to AS \mid AA \mid BA \mid \lambda \\ A \to aA \mid a \\ B \to bB \mid bC \mid C \\ C \to aA \mid bA \mid B \mid ab \end{cases}$$

1. *We execute the algorithm to find the unit productions.*

$$\begin{aligned} &\mathbf{UNIT_1}(S) = \{S\}. \\ &\mathbf{UNIT_2}(S) = \{S\} \cup \{\ \} = \{S\}. \\ &\mathbf{UNIT_1}(A) = \{A\}. \\ &\mathbf{UNIT_2}(A) = \{A\} \cup \{\ \} = \{A\}. \end{aligned}$$

Solution

$$G : \begin{cases} S \to AS \mid AA \mid BA \mid \lambda \\ A \to aA \mid a \\ B \to bB \mid bC \mid C \\ C \to aA \mid bA \mid B \mid ab \end{cases}$$

1. *We execute the algorithm to find the unit productions.*

$$\begin{aligned} &\mathbf{UNIT_1}(S) = \{S\}. \\ &\mathbf{UNIT_2}(S) = \{S\} \cup \{\ \} = \{S\}. \\ &\mathbf{UNIT_1}(A) = \{A\}. \\ &\mathbf{UNIT_2}(A) = \{A\} \cup \{\ \} = \{A\}. \\ &\mathbf{UNIT_1}(B) = \{B\}. \end{aligned}$$

Solution

$$G : \begin{cases} S \to AS \mid AA \mid BA \mid \lambda \\ A \to aA \mid a \\ B \to bB \mid bC \mid C \\ C \to aA \mid bA \mid B \mid ab \end{cases}$$

1. *We execute the algorithm to find the unit productions.*

$$\begin{aligned} &\mathbf{UNIT_1}(S) = \{S\}. \\ &\mathbf{UNIT_2}(S) = \{S\} \cup \{\ \} = \{S\}. \\ &\mathbf{UNIT_1}(A) = \{A\}. \\ &\mathbf{UNIT_2}(A) = \{A\} \cup \{\ \} = \{A\}. \\ &\mathbf{UNIT_1}(B) = \{B\}. \\ &\mathbf{UNIT_2}(B) = \{B\} \cup \{C\} = \{B, C\}. \end{aligned}$$

Solution

$$G : \begin{cases} S \to AS \mid AA \mid BA \mid \lambda \\ A \to aA \mid a \\ B \to bB \mid bC \mid C \\ C \to aA \mid bA \mid B \mid ab \end{cases}$$

1. *We execute the algorithm to find the unit productions.*

$$\begin{aligned} &\mathbf{UNIT_1}(S) = \{S\}. \\ &\mathbf{UNIT_2}(S) = \{S\} \cup \{\ \} = \{S\}. \\ &\mathbf{UNIT_1}(A) = \{A\}. \\ &\mathbf{UNIT_2}(A) = \{A\} \cup \{\ \} = \{A\}. \\ &\mathbf{UNIT_1}(B) = \{B\}. \\ &\mathbf{UNIT_2}(B) = \{B\} \cup \{C\} = \{B, C\}. \\ &\mathbf{UNIT_3}(B) = \{B, C\} \cup \{B\} = \{B, C\}. \end{aligned}$$

Solution

$$G : \begin{cases} S \to AS \mid AA \mid BA \mid \lambda \\ A \to aA \mid a \\ B \to bB \mid bC \mid C \\ C \to aA \mid bA \mid B \mid ab \end{cases}$$

1. *We execute the algorithm to find the unit productions.*

$$\begin{aligned}
&\mathbf{UNIT_1}(S) = \{S\}. \\
&\mathbf{UNIT_2}(S) = \{S\} \cup \{\ \} = \{S\}. \\
&\mathbf{UNIT_1}(A) = \{A\}. \\
&\mathbf{UNIT_2}(A) = \{A\} \cup \{\ \} = \{A\}. \\
&\mathbf{UNIT_1}(B) = \{B\}. \\
&\mathbf{UNIT_2}(B) = \{B\} \cup \{C\} = \{B, C\}. \\
&\mathbf{UNIT_3}(B) = \{B, C\} \cup \{B\} = \{B, C\}. \\
&\mathbf{UNIT_1}(C) = \{C\}.
\end{aligned}$$

Solution

$$G : \begin{cases} S \to AS \mid AA \mid BA \mid \lambda \\ A \to aA \mid a \\ B \to bB \mid bC \mid C \\ C \to aA \mid bA \mid B \mid ab \end{cases}$$

1. *We execute the algorithm to find the unit productions.*

$$\begin{aligned}
&\mathbf{UNIT_1}(S) = \{S\}. \\
&\mathbf{UNIT_2}(S) = \{S\} \cup \{\ \} = \{S\}. \\
&\mathbf{UNIT_1}(A) = \{A\}. \\
&\mathbf{UNIT_2}(A) = \{A\} \cup \{\ \} = \{A\}. \\
&\mathbf{UNIT_1}(B) = \{B\}. \\
&\mathbf{UNIT_2}(B) = \{B\} \cup \{C\} = \{B, C\}. \\
&\mathbf{UNIT_3}(B) = \{B, C\} \cup \{B\} = \{B, C\}. \\
&\mathbf{UNIT_1}(C) = \{C\}. \\
&\mathbf{UNIT_2}(C) = \{C\} \cup \{B\} = \{C, B\}.
\end{aligned}$$

Solution

$$G : \begin{cases} S \to AS \mid AA \mid BA \mid \lambda \\ A \to aA \mid a \\ B \to bB \mid bC \mid C \\ C \to aA \mid bA \mid B \mid ab \end{cases}$$

1. *We execute the algorithm to find the unit productions.*

$$\begin{aligned}
&\mathbf{UNIT_1}(S) = \{S\}. \\
&\mathbf{UNIT_2}(S) = \{S\} \cup \{\ \} = \{S\}. \\
&\mathbf{UNIT_1}(A) = \{A\}. \\
&\mathbf{UNIT_2}(A) = \{A\} \cup \{\ \} = \{A\}. \\
&\mathbf{UNIT_1}(B) = \{B\}. \\
&\mathbf{UNIT_2}(B) = \{B\} \cup \{C\} = \{B, C\}. \\
&\mathbf{UNIT_3}(B) = \{B, C\} \cup \{B\} = \{B, C\}. \\
&\mathbf{UNIT_1}(C) = \{C\}. \\
&\mathbf{UNIT_2}(C) = \{C\} \cup \{B\} = \{C, B\}. \\
&\mathbf{UNIT_3}(C) = \{C, B\} \cup \{C\} = \{C, B\}.
\end{aligned}$$

Solution

$$G : \begin{cases} S \to AS \mid AA \mid BA \mid \lambda \\ A \to aA \mid a \\ B \to bB \mid bC \mid C \\ C \to aA \mid bA \mid B \mid ab \end{cases}$$

1. *We execute the algorithm to find the unit productions.*

$$\begin{aligned}
&\mathbf{UNIT_1}(S) = \{S\}. \\
&\mathbf{UNIT_2}(S) = \{S\} \cup \{\ \} = \{S\}. \\
&\mathbf{UNIT_1}(A) = \{A\}. \\
&\mathbf{UNIT_2}(A) = \{A\} \cup \{\ \} = \{A\}. \\
&\mathbf{UNIT_1}(B) = \{B\}. \\
&\mathbf{UNIT_2}(B) = \{B\} \cup \{C\} = \{B, C\}. \\
&\mathbf{UNIT_3}(B) = \{B, C\} \cup \{B\} = \{B, C\}. \\
&\mathbf{UNIT_1}(C) = \{C\}. \\
&\mathbf{UNIT_2}(C) = \{C\} \cup \{B\} = \{C, B\}. \\
&\mathbf{UNIT_3}(C) = \{C, B\} \cup \{C\} = \{C, B\}.
\end{aligned}$$

2. *By removing the unit productions we obtain an equivalent grammar G' :*

Solution

$$G: \begin{cases} S \to AS \mid AA \mid BA \mid \lambda \\ A \to aA \mid a \\ B \to bB \mid bC \mid C \\ C \to aA \mid bA \mid B \mid ab \end{cases}$$

1. *We execute the algorithm to find the unit productions.*

$$\begin{aligned}
&\mathbf{UNIT_1}(S) = \{S\}. \\
&\mathbf{UNIT_2}(S) = \{S\} \cup \{\ \} = \{S\}. \\
&\mathbf{UNIT_1}(A) = \{A\}. \\
&\mathbf{UNIT_2}(A) = \{A\} \cup \{\ \} = \{A\}. \\
&\mathbf{UNIT_1}(B) = \{B\}. \\
&\mathbf{UNIT_2}(B) = \{B\} \cup \{C\} = \{B, C\}. \\
&\mathbf{UNIT_3}(B) = \{B, C\} \cup \{B\} = \{B, C\}. \\
&\mathbf{UNIT_1}(C) = \{C\}. \\
&\mathbf{UNIT_2}(C) = \{C\} \cup \{B\} = \{C, B\}. \\
&\mathbf{UNIT_3}(C) = \{C, B\} \cup \{C\} = \{C, B\}.
\end{aligned}$$

2. *By removing the unit productions we obtain an equivalent grammar G' :*

$$G' : \begin{cases} S \to AS \mid AA \mid BA \mid \lambda \\ \\ \\ \end{cases}$$

Solution

$$G: \begin{cases} S \to AS \mid AA \mid BA \mid \lambda \\ A \to aA \mid a \\ B \to bB \mid bC \mid C \\ C \to aA \mid bA \mid B \mid ab \end{cases}$$

1. *We execute the algorithm to find the unit productions.*

$$\begin{aligned}
&\mathbf{UNIT_1}(S) = \{S\}. \\
&\mathbf{UNIT_2}(S) = \{S\} \cup \{\ \} = \{S\}. \\
&\mathbf{UNIT_1}(A) = \{A\}. \\
&\mathbf{UNIT_2}(A) = \{A\} \cup \{\ \} = \{A\}. \\
&\mathbf{UNIT_1}(B) = \{B\}. \\
&\mathbf{UNIT_2}(B) = \{B\} \cup \{C\} = \{B, C\}. \\
&\mathbf{UNIT_3}(B) = \{B, C\} \cup \{B\} = \{B, C\}. \\
&\mathbf{UNIT_1}(C) = \{C\}. \\
&\mathbf{UNIT_2}(C) = \{C\} \cup \{B\} = \{C, B\}. \\
&\mathbf{UNIT_3}(C) = \{C, B\} \cup \{C\} = \{C, B\}.
\end{aligned}$$

2. *By removing the unit productions we obtain an equivalent grammar G' :*

$$G': \begin{cases} S \to AS \mid AA \mid BA \mid \lambda \\ A \to aA \mid a \end{cases}$$

Solution

$$G: \begin{cases} S \to AS \mid AA \mid BA \mid \lambda \\ A \to aA \mid a \\ B \to bB \mid bC \mid C \\ C \to aA \mid bA \mid B \mid ab \end{cases}$$

1. *We execute the algorithm to find the unit productions.*

$$\begin{aligned}
&\mathbf{UNIT_1}(S) = \{S\}. \\
&\mathbf{UNIT_2}(S) = \{S\} \cup \{\ \} = \{S\}. \\
&\mathbf{UNIT_1}(A) = \{A\}. \\
&\mathbf{UNIT_2}(A) = \{A\} \cup \{\ \} = \{A\}. \\
&\mathbf{UNIT_1}(B) = \{B\}. \\
&\mathbf{UNIT_2}(B) = \{B\} \cup \{C\} = \{B, C\}. \\
&\mathbf{UNIT_3}(B) = \{B, C\} \cup \{B\} = \{B, C\}. \\
&\mathbf{UNIT_1}(C) = \{C\}. \\
&\mathbf{UNIT_2}(C) = \{C\} \cup \{B\} = \{C, B\}. \\
&\mathbf{UNIT_3}(C) = \{C, B\} \cup \{C\} = \{C, B\}.
\end{aligned}$$

2. *By removing the unit productions we obtain an equivalent grammar G' :*

$$G': \begin{cases} S \to AS \mid AA \mid BA \mid \lambda \\ A \to aA \mid a \\ B \to bB \mid bC \mid aA \mid bA \mid ab \end{cases}$$

Solution

$$G: \begin{cases} S \to AS \mid AA \mid BA \mid \lambda \\ A \to aA \mid a \\ B \to bB \mid bC \mid C \\ C \to aA \mid bA \mid B \mid ab \end{cases}$$

1. *We execute the algorithm to find the unit productions.*

$$\begin{aligned}
&\mathbf{UNIT_1}(S) = \{S\}. \\
&\mathbf{UNIT_2}(S) = \{S\} \cup \{\ \} = \{S\}. \\
&\mathbf{UNIT_1}(A) = \{A\}. \\
&\mathbf{UNIT_2}(A) = \{A\} \cup \{\ \} = \{A\}. \\
&\mathbf{UNIT_1}(B) = \{B\}. \\
&\mathbf{UNIT_2}(B) = \{B\} \cup \{C\} = \{B, C\}. \\
&\mathbf{UNIT_3}(B) = \{B, C\} \cup \{B\} = \{B, C\}. \\
&\mathbf{UNIT_1}(C) = \{C\}. \\
&\mathbf{UNIT_2}(C) = \{C\} \cup \{B\} = \{C, B\}. \\
&\mathbf{UNIT_3}(C) = \{C, B\} \cup \{C\} = \{C, B\}.
\end{aligned}$$

2. *By removing the unit productions we obtain an equivalent grammar G' :*

$$G': \begin{cases} S \to AS \mid AA \mid BA \mid \lambda \\ A \to aA \mid a \\ B \to bB \mid bC \mid aA \mid bA \mid ab \\ C \to aA \mid bA \mid ab \mid bB \mid bC \end{cases}$$

When is a CFG G in Chomsky Normal Form?

When is a CFG G in Chomsky Normal Form?

1. G has no useless variables.

When is a CFG G in Chomsky Normal Form?

1. G has no useless variables.
2. G has no λ productions (except possibly $S \to \lambda$).

When is a CFG G in Chomsky Normal Form?

1. G has no useless variables.
2. G has no λ productions (except possibly $S \to \lambda$).
3. All productions are of the form: $A \to a$ or $A \to BC$ (binary productions).

When is a CFG G in Chomsky Normal Form?

1. G has no useless variables.
2. G has no λ productions (except possibly $S \to \lambda$).
3. All productions are of the form: $A \to a$ or $A \to BC$ (binary productions).

In particular, a grammar in CNF does not have unit productions.

When is a CFG G in Chomsky Normal Form?

1. G has no useless variables.
2. G has no λ productions (except possibly $S \to \lambda$).
3. All productions are of the form: $A \to a$ or $A \to BC$ (binary productions).

In particular, a grammar in CNF does not have unit productions.

Can a CFG G be converted to an equivalent grammar in CNF?

When is a CFG G in Chomsky Normal Form?

1. G has no useless variables.
2. G has no λ productions (except possibly $S \to \lambda$).
3. All productions are of the form: $A \to a$ or $A \to BC$ (binary productions).

In particular, a grammar in CNF does not have unit productions.

Can a CFG G be converted to an equivalent grammar in CNF?

Theorem

Every CFG G is equivalent to a grammar in Chomsky Normal Form.

How do you convert a CFG G to an equivalent grammar in CNF?

How do you convert a CFG G to an equivalent grammar in CNF?

1. Remove non-generating variables.

How do you convert a CFG G to an equivalent grammar in CNF?

1. Remove non-generating variables.
2. Remove non-reachable variables.

How do you convert a CFG G to an equivalent grammar in CNF?

1. Remove non-generating variables.
2. Remove non-reachable variables.
3. Remove λ productions (except possibly $S \to \lambda$).

How do you convert a CFG G to an equivalent grammar in CNF?

1. Remove non-generating variables.
2. Remove non-reachable variables.
3. Remove λ productions (except possibly $S \to \lambda$).
4. Eliminate unit productions.

How do you convert a CFG G to an equivalent grammar in CNF?

1. Remove non-generating variables.
2. Remove non-reachable variables.
3. Remove λ productions (except possibly $S \to \lambda$).
4. Eliminate unit productions.
5. The resulting productions (different from $S \to \lambda$) should be of the form $A \to a$ or of the form $A \to w$, where $a \in \Sigma$, $w \in V^*$ and $|w| \geq 2$. These last ones can be simulated with productions of the form $A \to BC$ or $A \to a$. First, introduce, for each $a \in \Sigma$, a new variable T_a with the only production $T_a \to a$. Then, introduce new variables, with binary productions, to simulate the desired productions.

How can a production be simulated using only simple and binary productions?

How can a production be simulated using only simple and binary productions?

Example

Simulating the production $A \rightarrow abBaC$ with binary and simple productions.

How can a production be simulated using only simple and binary productions?

Example

Simulating the production $A \to abBaC$ with binary and simple productions.

Solution

We introduce the variables T_a and T_b, and the productions $T_a \to a$ and $T_b \to b$. Then $A \to abBaC$ is simulated as:

How can a production be simulated using only simple and binary productions?

Example

Simulating the production $A \to abBaC$ with binary and simple productions.

Solution

We introduce the variables T_a and T_b, and the productions $T_a \to a$ and $T_b \to b$. Then $A \to abBaC$ is simulated as:

$$\begin{cases} A \to T_a T_b B T_a C \\ \\ \end{cases}$$

How can a production be simulated using only simple and binary productions?

Example

Simulating the production $A \to abBaC$ with binary and simple productions.

Solution

We introduce the variables T_a and T_b, and the productions $T_a \to a$ and $T_b \to b$. Then $A \to abBaC$ is simulated as:

$$\begin{cases} A \to T_a T_b B T_a C \\ t_a \to a \end{cases}$$

How can a production be simulated using only simple and binary productions?

Example

Simulating the production $A \to abBaC$ with binary and simple productions.

Solution

We introduce the variables T_a and T_b, and the productions $T_a \to a$ and $T_b \to b$. Then $A \to abBaC$ is simulated as:

$$\begin{cases} A \to T_a T_b B T_a C \\ t_a \to a \\ t_b \to b \end{cases}$$

How can a production be simulated using only simple and binary productions?

Example

Simulating the production $A \to abBaC$ with binary and simple productions.

Solution

We introduce the variables T_a and T_b, and the productions $T_a \to a$ and $T_b \to b$. Then $A \to abBaC$ is simulated as:

$$\begin{cases} A \to T_a T_b B T_a C \\ t_a \to a \\ t_b \to b \end{cases}$$

We introduce new variables T_1, T_2, T_3 and the necessary binary productions. The only productions for these new variables are as follows:

How can a production be simulated using only simple and binary productions?

Example

Simulating the production $A \to abBaC$ with binary and simple productions.

Solution

We introduce the variables T_a and T_b, and the productions $T_a \to a$ and $T_b \to b$. Then $A \to abBaC$ is simulated as:

$$\begin{cases} A \to T_a T_b B T_a C \\ t_a \to a \\ t_b \to b \end{cases}$$

We introduce new variables T_1, T_2, T_3 and the necessary binary productions. The only productions for these new variables are as follows:

$$\begin{cases} A \to T_a T_1 \\ \\ \\ \\ \end{cases}$$

How can a production be simulated using only simple and binary productions?

Example

Simulating the production $A \to abBaC$ with binary and simple productions.

Solution

We introduce the variables T_a and T_b, and the productions $T_a \to a$ and $T_b \to b$. Then $A \to abBaC$ is simulated as:

$$\begin{cases} A \to T_a T_b B T_a C \\ t_a \to a \\ t_b \to b \end{cases}$$

We introduce new variables T_1, T_2, T_3 and the necessary binary productions. The only productions for these new variables are as follows:

$$\begin{cases} A \to T_a T_1 \\ T_1 \to T_b T_2 \\ \\ \\ \end{cases}$$

How can a production be simulated using only simple and binary productions?

Example

Simulating the production $A \to abBaC$ with binary and simple productions.

Solution

We introduce the variables T_a and T_b, and the productions $T_a \to a$ and $T_b \to b$. Then $A \to abBaC$ is simulated as:

$$\begin{cases} A \to T_a T_b B T_a C \\ t_a \to a \\ t_b \to b \end{cases}$$

We introduce new variables T_1, T_2, T_3 and the necessary binary productions. The only productions for these new variables are as follows:

$$\begin{cases} A \to T_a T_1 \\ T_1 \to T_b T_2 \\ T_2 \to B T_3 \end{cases}$$

How can a production be simulated using only simple and binary productions?

Example

Simulating the production $A \to abBaC$ with binary and simple productions.

Solution

We introduce the variables T_a and T_b, and the productions $T_a \to a$ and $T_b \to b$. Then $A \to abBaC$ is simulated as:

$$\begin{cases} A \to T_a T_b B T_a C \\ t_a \to a \\ t_b \to b \end{cases}$$

We introduce new variables T_1, T_2, T_3 and the necessary binary productions. The only productions for these new variables are as follows:

$$\begin{cases} A \to T_a T_1 \\ T_1 \to T_b T_2 \\ T_2 \to B T_3 \\ T_3 \to T_a C \end{cases}$$

How can a production be simulated using only simple and binary productions?

Example

Simulating the production $A \to abBaC$ with binary and simple productions.

Solution

We introduce the variables T_a and T_b, and the productions $T_a \to a$ and $T_b \to b$. Then $A \to abBaC$ is simulated as:

$$\begin{cases} A \to T_a T_b B T_a C \\ t_a \to a \\ t_b \to b \end{cases}$$

We introduce new variables T_1, T_2, T_3 and the necessary binary productions. The only productions for these new variables are as follows:

$$\begin{cases} A \to T_a T_1 \\ T_1 \to T_b T_2 \\ T_2 \to B T_3 \\ T_3 \to T_a C \\ T_a \to a \end{cases}$$

How can a production be simulated using only simple and binary productions?

Example

Simulating the production $A \to abBaC$ with binary and simple productions.

Solution

We introduce the variables T_a and T_b, and the productions $T_a \to a$ and $T_b \to b$. Then $A \to abBaC$ is simulated as:

$$\begin{cases} A \to T_a T_b B T_a C \\ t_a \to a \\ t_b \to b \end{cases}$$

We introduce new variables T_1, T_2, T_3 and the necessary binary productions. The only productions for these new variables are as follows:

$$\begin{cases} A \to T_a T_1 \\ T_1 \to T_b T_2 \\ T_2 \to B T_3 \\ T_3 \to T_a C \\ T_a \to a \\ T_b \to b \end{cases}$$

Example

Find an equivalent CFG in CNF for the following grammar:

Example

Find an equivalent CFG in CNF for the following grammar:

$$\begin{cases} S \rightarrow AB \mid aBC \mid SBS \\ A \rightarrow aA \mid C \\ B \rightarrow bbB \mid b \\ C \rightarrow cC \mid \lambda \end{cases}$$

Example

Find an equivalent CFG in CNF for the following grammar:

$$\begin{cases} S \rightarrow AB \mid aBC \mid SBS \\ A \rightarrow aA \mid C \\ B \rightarrow bbB \mid b \\ C \rightarrow cC \mid \lambda \end{cases}$$

Solution

1. *We find the generating variables:*

Example

Find an equivalent CFG in CNF for the following grammar:

$$\begin{cases} S \to AB \mid aBC \mid SBS \\ A \to aA \mid C \\ B \to bbB \mid b \\ C \to cC \mid \lambda \end{cases}$$

Solution

1. *We find the generating variables:*

$$\mathbf{GEN_1} = \{B, C\}.$$

Example

Find an equivalent CFG in CNF for the following grammar:

$$\begin{cases} S \to AB \mid aBC \mid SBS \\ A \to aA \mid C \\ B \to bbB \mid b \\ C \to cC \mid \lambda \end{cases}$$

Solution

1. *We find the generating variables:*

$$\begin{aligned} \mathbf{GEN_1} &= \{B, C\}. \\ \mathbf{GEN_2} &= \{B, C\} \cup \{A, S\} = \{B, C, A, S\}. \end{aligned}$$

Example

Find an equivalent CFG in CNF for the following grammar:

$$\begin{cases} S \to AB \mid aBC \mid SBS \\ A \to aA \mid C \\ B \to bbB \mid b \\ C \to cC \mid \lambda \end{cases}$$

Solution

1. *We find the generating variables:*

$$\begin{aligned} \mathbf{GEN_1} &= \{B, C\}. \\ \mathbf{GEN_2} &= \{B, C\} \cup \{A, S\} = \{B, C, A, S\}. \\ \mathbf{GEN_3} &= \{B, C, A, S\} \cup \{\ \} = \{B, C, A, S\}. \end{aligned}$$

Example

Find an equivalent CFG in CNF for the following grammar:

$$\begin{cases} S \to AB \mid aBC \mid SBS \\ A \to aA \mid C \\ B \to bbB \mid b \\ C \to cC \mid \lambda \end{cases}$$

Solution

1. *We find the generating variables:*

$$\begin{aligned} \mathbf{GEN_1} &= \{B, C\}. \\ \mathbf{GEN_2} &= \{B, C\} \cup \{A, S\} = \{B, C, A, S\}. \\ \mathbf{GEN_3} &= \{B, C, A, S\} \cup \{\ \} = \{B, C, A, S\}. \end{aligned}$$

We find the reachable variables:

Example

Find an equivalent CFG in CNF for the following grammar:

$$\begin{cases} S \to AB \mid aBC \mid SBS \\ A \to aA \mid C \\ B \to bbB \mid b \\ C \to cC \mid \lambda \end{cases}$$

Solution

1. *We find the generating variables:*

$$\begin{aligned} \mathbf{GEN_1} &= \{B, C\}. \\ \mathbf{GEN_2} &= \{B, C\} \cup \{A, S\} = \{B, C, A, S\}. \\ \mathbf{GEN_3} &= \{B, C, A, S\} \cup \{\ \} = \{B, C, A, S\}. \end{aligned}$$

We find the reachable variables:

$$\mathbf{REACH_1} = \{S\}.$$

Example

Find an equivalent CFG in CNF for the following grammar:

$$\begin{cases} S \to AB \mid aBC \mid SBS \\ A \to aA \mid C \\ B \to bbB \mid b \\ C \to cC \mid \lambda \end{cases}$$

Solution

1. *We find the generating variables:*

$$\begin{aligned} \mathbf{GEN_1} &= \{B, C\}. \\ \mathbf{GEN_2} &= \{B, C\} \cup \{A, S\} = \{B, C, A, S\}. \\ \mathbf{GEN_3} &= \{B, C, A, S\} \cup \{\ \} = \{B, C, A, S\}. \end{aligned}$$

We find the reachable variables:

$$\begin{aligned} \mathbf{REACH_1} &= \{S\}. \\ \mathbf{REACH_2} &= \{S\} \cup \{A, B, C\} = \{S, A, B, C\}. \end{aligned}$$

Example

Find an equivalent CFG in CNF for the following grammar:

$$\begin{cases} S \to AB \mid aBC \mid SBS \\ A \to aA \mid C \\ B \to bbB \mid b \\ C \to cC \mid \lambda \end{cases}$$

Solution

1. *We find the generating variables:*

$$\begin{aligned} \mathbf{GEN_1} &= \{B, C\}. \\ \mathbf{GEN_2} &= \{B, C\} \cup \{A, S\} = \{B, C, A, S\}. \\ \mathbf{GEN_3} &= \{B, C, A, S\} \cup \{\ \} = \{B, C, A, S\}. \end{aligned}$$

We find the reachable variables:

$$\begin{aligned} \mathbf{REACH_1} &= \{S\}. \\ \mathbf{REACH_2} &= \{S\} \cup \{A, B, C\} = \{S, A, B, C\}. \\ \mathbf{REACH_3} &= \{S, A, B, C\} \cup \{\ \} = \{S, A, B, C\}. \end{aligned}$$

Example

Find an equivalent CFG in CNF for the following grammar:

$$\begin{cases} S \to AB \mid aBC \mid SBS \\ A \to aA \mid C \\ B \to bbB \mid b \\ C \to cC \mid \lambda \end{cases}$$

Solution

1. *We find the generating variables:*

$$\begin{aligned} \mathbf{GEN_1} &= \{B, C\}. \\ \mathbf{GEN_2} &= \{B, C\} \cup \{A, S\} = \{B, C, A, S\}. \\ \mathbf{GEN_3} &= \{B, C, A, S\} \cup \{\ \} = \{B, C, A, S\}. \end{aligned}$$

We find the reachable variables:

$$\begin{aligned} \mathbf{REACH_1} &= \{S\}. \\ \mathbf{REACH_2} &= \{S\} \cup \{A, B, C\} = \{S, A, B, C\}. \\ \mathbf{REACH_3} &= \{S, A, B, C\} \cup \{\ \} = \{S, A, B, C\}. \end{aligned}$$

As in this case there are no useless variables, we do not remove any variables.

Solution

$$\begin{cases} S \to AB \mid aBC \mid SBS \\ A \to aA \mid C \\ B \to bbB \mid b \\ C \to cC \mid \lambda \end{cases}$$

Solution

$$\begin{cases} S \to AB \mid aBC \mid SBS \\ A \to aA \mid C \\ B \to bbB \mid b \\ C \to cC \mid \lambda \end{cases}$$

2. *We find the nullable variables:*

$$\mathbf{NULL_1} = \{C\}.$$

Solution

$$\begin{cases} S \to AB \mid aBC \mid SBS \\ A \to aA \mid C \\ B \to bbB \mid b \\ C \to cC \mid \lambda \end{cases}$$

2. *We find the nullable variables:*

$$\mathbf{NULL_1} = \{C\}.$$
$$\mathbf{NULL_2} = \{C\} \cup \{A\} = \{C, A\}.$$

Solution

$$\begin{cases} S \to AB \mid aBC \mid SBS \\ A \to aA \mid C \\ B \to bbB \mid b \\ C \to cC \mid \lambda \end{cases}$$

2. *We find the nullable variables:*

$$\begin{aligned} \mathbf{NULL_1} &= \{C\}. \\ \mathbf{NULL_2} &= \{C\} \cup \{A\} = \{C, A\}. \\ \mathbf{NULL_3} &= \{C, A\} \cup \{\,\} = \{C, A\}. \end{aligned}$$

Solution

$$\begin{cases} S \to AB \mid aBC \mid SBS \\ A \to aA \mid C \\ B \to bbB \mid b \\ C \to cC \mid \lambda \end{cases}$$

2. *We find the nullable variables:*

$$\begin{aligned} \mathbf{NULL_1} &= \{C\}. \\ \mathbf{NULL_2} &= \{C\} \cup \{A\} = \{C, A\}. \\ \mathbf{NULL_3} &= \{C, A\} \cup \{\,\} = \{C, A\}. \end{aligned}$$

We remove the λ productions and add new productions that simulate the eliminated λ productions, obtaining the equivalent grammar G_1:

Solution

$$\begin{cases} S \to AB \mid aBC \mid SBS \\ A \to aA \mid C \\ B \to bbB \mid b \\ C \to cC \mid \lambda \end{cases}$$

2. *We find the nullable variables:*

$$\begin{aligned} \mathbf{NULL_1} &= \{C\}. \\ \mathbf{NULL_2} &= \{C\} \cup \{A\} = \{C, A\}. \\ \mathbf{NULL_3} &= \{C, A\} \cup \{\,\} = \{C, A\}. \end{aligned}$$

We remove the λ productions and add new productions that simulate the eliminated λ productions, obtaining the equivalent grammar G_1:

$$G_1 : \begin{cases} S \to AB \mid aBC \mid SBS \mid B \mid aB \\ \\ \\ \end{cases}$$

Solution

$$\begin{cases} S \to AB \mid aBC \mid SBS \\ A \to aA \mid C \\ B \to bbB \mid b \\ C \to cC \mid \lambda \end{cases}$$

2. *We find the nullable variables:*

$$\begin{aligned} \mathbf{NULL_1} &= \{C\}. \\ \mathbf{NULL_2} &= \{C\} \cup \{A\} = \{C, A\}. \\ \mathbf{NULL_3} &= \{C, A\} \cup \{\,\} = \{C, A\}. \end{aligned}$$

We remove the λ productions and add new productions that simulate the eliminated λ productions, obtaining the equivalent grammar G_1:

$$G_1 : \begin{cases} S \to AB \mid aBC \mid SBS \mid B \mid aB \\ A \to aA \mid C \mid a \\ \\ \end{cases}$$

Solution

$$\begin{cases} S \to AB \mid aBC \mid SBS \\ A \to aA \mid C \\ B \to bbB \mid b \\ C \to cC \mid \lambda \end{cases}$$

2. *We find the nullable variables:*

$$\begin{aligned} \mathbf{NULL_1} &= \{C\}. \\ \mathbf{NULL_2} &= \{C\} \cup \{A\} = \{C, A\}. \\ \mathbf{NULL_3} &= \{C, A\} \cup \{\,\} = \{C, A\}. \end{aligned}$$

We remove the λ productions and add new productions that simulate the eliminated λ productions, obtaining the equivalent grammar G_1:

$$G_1 : \begin{cases} S \to AB \mid aBC \mid SBS \mid B \mid aB \\ A \to aA \mid C \mid a \\ B \to bbB \mid b \end{cases}$$

Solution

$$\begin{cases} S \to AB \mid aBC \mid SBS \\ A \to aA \mid C \\ B \to bbB \mid b \\ C \to cC \mid \lambda \end{cases}$$

2. *We find the nullable variables:*

$$\begin{aligned} \mathbf{NULL_1} &= \{C\}. \\ \mathbf{NULL_2} &= \{C\} \cup \{A\} = \{C, A\}. \\ \mathbf{NULL_3} &= \{C, A\} \cup \{\,\} = \{C, A\}. \end{aligned}$$

We remove the λ productions and add new productions that simulate the eliminated λ productions, obtaining the equivalent grammar G_1:

$$G_1 : \begin{cases} S \to AB \mid aBC \mid SBS \mid B \mid aB \\ A \to aA \mid C \mid a \\ B \to bbB \mid b \\ C \to cC \mid c \end{cases}$$

Solution

$$G_1 : \begin{cases} S \rightarrow AB \mid aBC \mid SBS \mid B \mid aB \\ A \rightarrow aA \mid C \mid a \\ B \rightarrow bbB \mid b \\ C \rightarrow cC \mid c \end{cases}$$

Solution

$$G_1 : \begin{cases} S \rightarrow AB \mid aBC \mid SBS \mid B \mid aB \\ A \rightarrow aA \mid C \mid a \\ B \rightarrow bbB \mid b \\ C \rightarrow cC \mid c \end{cases}$$

3. *We compute the unit sets for each of the variables in G_1 :*

Solution

$$G_1 : \begin{cases} S \rightarrow AB \mid aBC \mid SBS \mid B \mid aB \\ A \rightarrow aA \mid C \mid a \\ B \rightarrow bbB \mid b \\ C \rightarrow cC \mid c \end{cases}$$

3. *We compute the unit sets for each of the variables in G_1:*

$$\mathbf{UNIT_1}(S) = \{S\}.$$

Solution

$$G_1 : \begin{cases} S \rightarrow AB \mid aBC \mid SBS \mid B \mid aB \\ A \rightarrow aA \mid C \mid a \\ B \rightarrow bbB \mid b \\ C \rightarrow cC \mid c \end{cases}$$

3. *We compute the unit sets for each of the variables in G_1:*

$$\mathbf{UNIT_1}(S) = \{S\}. \ \mathbf{UNIT_2}(S) = \{S\} \cup \{B\} = \{S, B\}.$$

Solution

$$G_1 : \begin{cases} S \rightarrow AB \mid aBC \mid SBS \mid B \mid aB \\ A \rightarrow aA \mid C \mid a \\ B \rightarrow bbB \mid b \\ C \rightarrow cC \mid c \end{cases}$$

3. *We compute the unit sets for each of the variables in G_1:*

$$\mathbf{UNIT_1}(S) = \{S\}. \ \mathbf{UNIT_2}(S) = \{S\} \cup \{B\} = \{S, B\}. \ \mathbf{UNIT_3}(S) = \{S, B\} \cup \{\ \} = \{S, B\}.$$

Solution

$$G_1 : \begin{cases} S \rightarrow AB \mid aBC \mid SBS \mid B \mid aB \\ A \rightarrow aA \mid C \mid a \\ B \rightarrow bbB \mid b \\ C \rightarrow cC \mid c \end{cases}$$

3. *We compute the unit sets for each of the variables in G_1:*

$\mathbf{UNIT_1}(S) = \{S\}$. $\mathbf{UNIT_2}(S) = \{S\} \cup \{B\} = \{S, B\}$. $\mathbf{UNIT_3}(S) = \{S, B\} \cup \{\ \} = \{S, B\}$.
$\mathbf{UNIT_1}(A) = \{A\}$.

Solution

$$G_1 : \begin{cases} S \rightarrow AB \mid aBC \mid SBS \mid B \mid aB \\ A \rightarrow aA \mid C \mid a \\ B \rightarrow bbB \mid b \\ C \rightarrow cC \mid c \end{cases}$$

3. *We compute the unit sets for each of the variables in G_1:*

$$\mathbf{UNIT_1}(S) = \{S\}. \ \mathbf{UNIT_2}(S) = \{S\} \cup \{B\} = \{S, B\}. \ \mathbf{UNIT_3}(S) = \{S, B\} \cup \{\ \} = \{S, B\}.$$
$$\mathbf{UNIT_1}(A) = \{A\}. \ \mathbf{UNIT_2}(A) = \{A\} \cup \{C\} = \{A, C\}.$$

Solution

$$G_1 : \begin{cases} S \rightarrow AB \mid aBC \mid SBS \mid B \mid aB \\ A \rightarrow aA \mid C \mid a \\ B \rightarrow bbB \mid b \\ C \rightarrow cC \mid c \end{cases}$$

3. *We compute the unit sets for each of the variables in G_1:*

$$\mathbf{UNIT_1}(S) = \{S\}. \ \mathbf{UNIT_2}(S) = \{S\} \cup \{B\} = \{S, B\}. \ \mathbf{UNIT_3}(S) = \{S, B\} \cup \{\ \} = \{S, B\}.$$
$$\mathbf{UNIT_1}(A) = \{A\}. \ \mathbf{UNIT_2}(A) = \{A\} \cup \{C\} = \{A, C\}. \ \mathbf{UNIT_3}(A) = \{A, C\} \cup \{\ \} = \{A, C\}.$$

Solution

$$G_1 : \begin{cases} S \rightarrow AB \mid aBC \mid SBS \mid B \mid aB \\ A \rightarrow aA \mid C \mid a \\ B \rightarrow bbB \mid b \\ C \rightarrow cC \mid c \end{cases}$$

3. *We compute the unit sets for each of the variables in G_1:*

$$\mathbf{UNIT_1}(S) = \{S\}. \ \mathbf{UNIT_2}(S) = \{S\} \cup \{B\} = \{S, B\}. \ \mathbf{UNIT_3}(S) = \{S, B\} \cup \{\ \} = \{S, B\}.$$
$$\mathbf{UNIT_1}(A) = \{A\}. \ \mathbf{UNIT_2}(A) = \{A\} \cup \{C\} = \{A, C\}. \ \mathbf{UNIT_3}(A) = \{A, C\} \cup \{\ \} = \{A, C\}.$$
$$\mathbf{UNIT_1}(B) = \{B\}.$$

Solution

$$G_1 : \begin{cases} S \rightarrow AB \mid aBC \mid SBS \mid B \mid aB \\ A \rightarrow aA \mid C \mid a \\ B \rightarrow bbB \mid b \\ C \rightarrow cC \mid c \end{cases}$$

3. *We compute the unit sets for each of the variables in G_1:*

$$\mathbf{UNIT_1}(S) = \{S\}. \ \mathbf{UNIT_2}(S) = \{S\} \cup \{B\} = \{S, B\}. \ \mathbf{UNIT_3}(S) = \{S, B\} \cup \{\ \} = \{S, B\}.$$
$$\mathbf{UNIT_1}(A) = \{A\}. \ \mathbf{UNIT_2}(A) = \{A\} \cup \{C\} = \{A, C\}. \ \mathbf{UNIT_3}(A) = \{A, C\} \cup \{\ \} = \{A, C\}.$$
$$\mathbf{UNIT_1}(B) = \{B\}. \ \mathbf{UNIT_2}(B) = \{B\} \cup \{\ \} = \{B\}.$$

Solution

$$G_1 : \begin{cases} S \rightarrow AB \mid aBC \mid SBS \mid B \mid aB \\ A \rightarrow aA \mid C \mid a \\ B \rightarrow bbB \mid b \\ C \rightarrow cC \mid c \end{cases}$$

3. *We compute the unit sets for each of the variables in G_1:*

$$\begin{aligned} &\mathbf{UNIT_1}(S) = \{S\}. \ \mathbf{UNIT_2}(S) = \{S\} \cup \{B\} = \{S, B\}. \ \mathbf{UNIT_3}(S) = \{S, B\} \cup \{\ \} = \{S, B\}. \\ &\mathbf{UNIT_1}(A) = \{A\}. \ \mathbf{UNIT_2}(A) = \{A\} \cup \{C\} = \{A, C\}. \ \mathbf{UNIT_3}(A) = \{A, C\} \cup \{\ \} = \{A, C\}. \\ &\mathbf{UNIT_1}(B) = \{B\}. \ \mathbf{UNIT_2}(B) = \{B\} \cup \{\ \} = \{B\}. \\ &\mathbf{UNIT_1}(C) = \{C\}. \end{aligned}$$

Solution

$$G_1 : \begin{cases} S \rightarrow AB \mid aBC \mid SBS \mid B \mid aB \\ A \rightarrow aA \mid C \mid a \\ B \rightarrow bbB \mid b \\ C \rightarrow cC \mid c \end{cases}$$

3. *We compute the unit sets for each of the variables in G_1:*

$$\mathbf{UNIT_1}(S) = \{S\}.\ \mathbf{UNIT_2}(S) = \{S\} \cup \{B\} = \{S, B\}.\ \mathbf{UNIT_3}(S) = \{S, B\} \cup \{\ \} = \{S, B\}.$$
$$\mathbf{UNIT_1}(A) = \{A\}.\ \mathbf{UNIT_2}(A) = \{A\} \cup \{C\} = \{A, C\}.\ \mathbf{UNIT_3}(A) = \{A, C\} \cup \{\ \} = \{A, C\}.$$
$$\mathbf{UNIT_1}(B) = \{B\}.\ \mathbf{UNIT_2}(B) = \{B\} \cup \{\ \} = \{B\}.$$
$$\mathbf{UNIT_1}(C) = \{C\}.\ \mathbf{UNIT_2}(C) = \{C\} \cup \{\ \} = \{C\}.$$

Solution

$$G_1 : \begin{cases} S \to AB \mid aBC \mid SBS \mid B \mid aB \\ A \to aA \mid C \mid a \\ B \to bbB \mid b \\ C \to cC \mid c \end{cases}$$

3. *We compute the unit sets for each of the variables in G_1:*

$$\begin{aligned} &\mathbf{UNIT_1}(S) = \{S\}. \ \mathbf{UNIT_2}(S) = \{S\} \cup \{B\} = \{S, B\}. \ \mathbf{UNIT_3}(S) = \{S, B\} \cup \{\ \} = \{S, B\}. \\ &\mathbf{UNIT_1}(A) = \{A\}. \ \mathbf{UNIT_2}(A) = \{A\} \cup \{C\} = \{A, C\}. \ \mathbf{UNIT_3}(A) = \{A, C\} \cup \{\ \} = \{A, C\}. \\ &\mathbf{UNIT_1}(B) = \{B\}. \ \mathbf{UNIT_2}(B) = \{B\} \cup \{\ \} = \{B\}. \\ &\mathbf{UNIT_1}(C) = \{C\}. \ \mathbf{UNIT_2}(C) = \{C\} \cup \{\ \} = \{C\}. \end{aligned}$$

We remove the unit productions and add, for each variable A in G, the (non-unit) productions of the variables contained in the unit set $UNIT(A)$, obtaining the equivalent grammar G_2:

Solution

$$G_1 : \begin{cases} S \to AB \mid aBC \mid SBS \mid B \mid aB \\ A \to aA \mid C \mid a \\ B \to bbB \mid b \\ C \to cC \mid c \end{cases}$$

3. *We compute the unit sets for each of the variables in G_1:*

$$\begin{array}{l} \mathbf{UNIT_1}(S) = \{S\}.\ \mathbf{UNIT_2}(S) = \{S\} \cup \{B\} = \{S, B\}.\ \mathbf{UNIT_3}(S) = \{S, B\} \cup \{\ \} = \{S, B\}. \\ \mathbf{UNIT_1}(A) = \{A\}.\ \mathbf{UNIT_2}(A) = \{A\} \cup \{C\} = \{A, C\}.\ \mathbf{UNIT_3}(A) = \{A, C\} \cup \{\ \} = \{A, C\}. \\ \mathbf{UNIT_1}(B) = \{B\}.\ \mathbf{UNIT_2}(B) = \{B\} \cup \{\ \} = \{B\}. \\ \mathbf{UNIT_1}(C) = \{C\}.\ \mathbf{UNIT_2}(C) = \{C\} \cup \{\ \} = \{C\}. \end{array}$$

We remove the unit productions and add, for each variable A in G, the (non-unit) productions of the variables contained in the unit set $UNIT(A)$, obtaining the equivalent grammar G_2:

$$G_2 : \begin{cases} S \to AB \mid aBC \mid SBS \mid aB \mid bbB \mid b \\ \\ \\ \end{cases}$$

Solution

$$G_1 : \begin{cases} S \to AB \mid aBC \mid SBS \mid B \mid aB \\ A \to aA \mid C \mid a \\ B \to bbB \mid b \\ C \to cC \mid c \end{cases}$$

3. *We compute the unit sets for each of the variables in G_1:*

$$\begin{array}{l} \mathbf{UNIT_1}(S) = \{S\}.\ \mathbf{UNIT_2}(S) = \{S\} \cup \{B\} = \{S, B\}.\ \mathbf{UNIT_3}(S) = \{S, B\} \cup \{\ \} = \{S, B\}. \\ \mathbf{UNIT_1}(A) = \{A\}.\ \mathbf{UNIT_2}(A) = \{A\} \cup \{C\} = \{A, C\}.\ \mathbf{UNIT_3}(A) = \{A, C\} \cup \{\ \} = \{A, C\}. \\ \mathbf{UNIT_1}(B) = \{B\}.\ \mathbf{UNIT_2}(B) = \{B\} \cup \{\ \} = \{B\}. \\ \mathbf{UNIT_1}(C) = \{C\}.\ \mathbf{UNIT_2}(C) = \{C\} \cup \{\ \} = \{C\}. \end{array}$$

We remove the unit productions and add, for each variable A in G, the (non-unit) productions of the variables contained in the unit set $UNIT(A)$, obtaining the equivalent grammar G_2:

$$G_2 : \begin{cases} S \to AB \mid aBC \mid SBS \mid aB \mid bbB \mid b \\ A \to aA \mid a \mid cC \mid c \end{cases}$$

Solution

$$G_1 : \begin{cases} S \to AB \mid aBC \mid SBS \mid B \mid aB \\ A \to aA \mid C \mid a \\ B \to bbB \mid b \\ C \to cC \mid c \end{cases}$$

3. *We compute the unit sets for each of the variables in G_1:*

$$\begin{array}{l} \mathbf{UNIT_1}(S) = \{S\}.\ \mathbf{UNIT_2}(S) = \{S\} \cup \{B\} = \{S, B\}.\ \mathbf{UNIT_3}(S) = \{S, B\} \cup \{\ \} = \{S, B\}. \\ \mathbf{UNIT_1}(A) = \{A\}.\ \mathbf{UNIT_2}(A) = \{A\} \cup \{C\} = \{A, C\}.\ \mathbf{UNIT_3}(A) = \{A, C\} \cup \{\ \} = \{A, C\}. \\ \mathbf{UNIT_1}(B) = \{B\}.\ \mathbf{UNIT_2}(B) = \{B\} \cup \{\ \} = \{B\}. \\ \mathbf{UNIT_1}(C) = \{C\}.\ \mathbf{UNIT_2}(C) = \{C\} \cup \{\ \} = \{C\}. \end{array}$$

We remove the unit productions and add, for each variable A in G, the (non-unit) productions of the variables contained in the unit set $UNIT(A)$, obtaining the equivalent grammar G_2:

$$G_2 : \begin{cases} S \to AB \mid aBC \mid SBS \mid aB \mid bbB \mid b \\ A \to aA \mid a \mid cC \mid c \\ B \to bbB \mid b \end{cases}$$

Solution

$$G_1 : \begin{cases} S \to AB \mid aBC \mid SBS \mid B \mid aB \\ A \to aA \mid C \mid a \\ B \to bbB \mid b \\ C \to cC \mid c \end{cases}$$

3. *We compute the unit sets for each of the variables in G_1:*

$$\begin{array}{l} \mathbf{UNIT_1}(S) = \{S\}.\ \mathbf{UNIT_2}(S) = \{S\} \cup \{B\} = \{S, B\}.\ \mathbf{UNIT_3}(S) = \{S, B\} \cup \{\ \} = \{S, B\}. \\ \mathbf{UNIT_1}(A) = \{A\}.\ \mathbf{UNIT_2}(A) = \{A\} \cup \{C\} = \{A, C\}.\ \mathbf{UNIT_3}(A) = \{A, C\} \cup \{\ \} = \{A, C\}. \\ \mathbf{UNIT_1}(B) = \{B\}.\ \mathbf{UNIT_2}(B) = \{B\} \cup \{\ \} = \{B\}. \\ \mathbf{UNIT_1}(C) = \{C\}.\ \mathbf{UNIT_2}(C) = \{C\} \cup \{\ \} = \{C\}. \end{array}$$

We remove the unit productions and add, for each variable A in G, the (non-unit) productions of the variables contained in the unit set $UNIT(A)$, obtaining the equivalent grammar G_2:

$$G_2 : \begin{cases} S \to AB \mid aBC \mid SBS \mid aB \mid bbB \mid b \\ A \to aA \mid a \mid cC \mid c \\ B \to bbB \mid b \\ C \to cC \mid c \end{cases}$$

Solution

$$G_2 : \begin{cases} S \to AB \mid aBC \mid SBS \mid aB \mid bbB \mid b \\ A \to aA \mid a \mid cC \mid c \\ B \to bbB \mid b \\ C \to cC \mid c \end{cases}$$

Solution

$$G_2 : \begin{cases} S \to AB \mid aBC \mid SBS \mid aB \mid bbB \mid b \\ A \to aA \mid a \mid cC \mid c \\ B \to bbB \mid b \\ C \to cC \mid c \end{cases}$$

4. *We introduce new variables for each $a \in \Sigma$: T_a, T_b, T_c and the new transitions: $T_a \to a, T_b \to b, T_c \to c$. This is done so that all productions are of the form $A \to a$ or of the form $A \to w$, where $a \in \Sigma$, $w \in V^*$ and $|w| \geq 2$. Thus, we arrive at the grammar G_3 :*

Solution

$$G_2 : \begin{cases} S \to AB \mid aBC \mid SBS \mid aB \mid bbB \mid b \\ A \to aA \mid a \mid cC \mid c \\ B \to bbB \mid b \\ C \to cC \mid c \end{cases}$$

4. *We introduce new variables for each $a \in \Sigma$: T_a, T_b, T_c and the new transitions: $T_a \to a, T_b \to b, T_c \to c$. This is done so that all productions are of the form $A \to a$ or of the form $A \to w$, where $a \in \Sigma$, $w \in V^*$ and $|w| \geq 2$. Thus, we arrive at the grammar G_3 :*

$$G_3 : \begin{cases} S \to AB \mid T_aBC \mid SBS \mid T_aB \mid T_bT_bB \mid b \\ \\ \\ \\ \end{cases}$$

Solution

$$G_2 : \begin{cases} S \to AB \mid aBC \mid SBS \mid aB \mid bbB \mid b \\ A \to aA \mid a \mid cC \mid c \\ B \to bbB \mid b \\ C \to cC \mid c \end{cases}$$

4. *We introduce new variables for each $a \in \Sigma$: T_a, T_b, T_c and the new transitions: $T_a \to a, T_b \to b, T_c \to c$. This is done so that all productions are of the form $A \to a$ or of the form $A \to w$, where $a \in \Sigma$, $w \in V^*$ and $|w| \geq 2$. Thus, we arrive at the grammar G_3 :*

$$G_3 : \begin{cases} S \to AB \mid T_aBC \mid SBS \mid T_aB \mid T_bT_bB \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \end{cases}$$

Solution

$$G_2 : \begin{cases} S \to AB \mid aBC \mid SBS \mid aB \mid bbB \mid b \\ A \to aA \mid a \mid cC \mid c \\ B \to bbB \mid b \\ C \to cC \mid c \end{cases}$$

4. *We introduce new variables for each $a \in \Sigma$: T_a, T_b, T_c and the new transitions: $T_a \to a, T_b \to b, T_c \to c$. This is done so that all productions are of the form $A \to a$ or of the form $A \to w$, where $a \in \Sigma$, $w \in V^*$ and $|w| \geq 2$. Thus, we arrive at the grammar G_3 :*

$$G_3 : \begin{cases} S \to AB \mid T_aBC \mid SBS \mid T_aB \mid T_bT_bB \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_bB \mid b \end{cases}$$

Solution

$$G_2 : \begin{cases} S \to AB \mid aBC \mid SBS \mid aB \mid bbB \mid b \\ A \to aA \mid a \mid cC \mid c \\ B \to bbB \mid b \\ C \to cC \mid c \end{cases}$$

4. *We introduce new variables for each $a \in \Sigma$: T_a, T_b, T_c and the new transitions: $T_a \to a, T_b \to b, T_c \to c$. This is done so that all productions are of the form $A \to a$ or of the form $A \to w$, where $a \in \Sigma$, $w \in V^*$ and $|w| \geq 2$. Thus, we arrive at the grammar G_3 :*

$$G_3 : \begin{cases} S \to AB \mid T_aBC \mid SBS \mid T_aB \mid T_bT_bB \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_bB \mid b \\ C \to T_cC \mid c \end{cases}$$

Solution

$$G_2 : \begin{cases} S \to AB \mid aBC \mid SBS \mid aB \mid bbB \mid b \\ A \to aA \mid a \mid cC \mid c \\ B \to bbB \mid b \\ C \to cC \mid c \end{cases}$$

4. *We introduce new variables for each $a \in \Sigma$: T_a, T_b, T_c and the new transitions: $T_a \to a, T_b \to b, T_c \to c$. This is done so that all productions are of the form $A \to a$ or of the form $A \to w$, where $a \in \Sigma$, $w \in V^*$ and $|w| \geq 2$. Thus, we arrive at the grammar G_3 :*

$$G_3 : \begin{cases} S \to AB \mid T_aBC \mid SBS \mid T_aB \mid T_bT_bB \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_bB \mid b \\ C \to T_cC \mid c \\ T_a \to a \end{cases}$$

Solution

$$G_2 : \begin{cases} S \to AB \mid aBC \mid SBS \mid aB \mid bbB \mid b \\ A \to aA \mid a \mid cC \mid c \\ B \to bbB \mid b \\ C \to cC \mid c \end{cases}$$

4. *We introduce new variables for each $a \in \Sigma$: T_a, T_b, T_c and the new transitions: $T_a \to a, T_b \to b, T_c \to c$. This is done so that all productions are of the form $A \to a$ or of the form $A \to w$, where $a \in \Sigma$, $w \in V^*$ and $|w| \geq 2$. Thus, we arrive at the grammar G_3 :*

$$G_3 : \begin{cases} S \to AB \mid T_aBC \mid SBS \mid T_aB \mid T_bT_bB \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_bB \mid b \\ C \to T_cC \mid c \\ T_a \to a \\ T_b \to b \end{cases}$$

Solution

$$G_2 : \begin{cases} S \to AB \mid aBC \mid SBS \mid aB \mid bbB \mid b \\ A \to aA \mid a \mid cC \mid c \\ B \to bbB \mid b \\ C \to cC \mid c \end{cases}$$

4. *We introduce new variables for each $a \in \Sigma$: T_a, T_b, T_c and the new transitions: $T_a \to a, T_b \to b, T_c \to c$. This is done so that all productions are of the form $A \to a$ or of the form $A \to w$, where $a \in \Sigma$, $w \in V^*$ and $|w| \geq 2$. Thus, we arrive at the grammar G_3 :*

$$G_3 : \begin{cases} S \to AB \mid T_aBC \mid SBS \mid T_aB \mid T_bT_bB \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_bB \mid b \\ C \to T_cC \mid c \\ T_a \to a \\ T_b \to b \\ T_c \to c \end{cases}$$

Solution

$$G_3 : \begin{cases} S \rightarrow AB \mid T_aBC \mid SBS \mid T_aB \mid T_bT_bB \mid b \\ A \rightarrow T_aA \mid a \mid T_cC \mid c \\ B \rightarrow T_bT_bB \mid b \\ C \rightarrow T_cC \mid c \\ T_a \rightarrow a \\ T_b \rightarrow b \\ T_c \rightarrow c \end{cases}$$

Solution

$$G_3 : \begin{cases} S \to AB \mid T_aBC \mid SBS \mid T_aB \mid T_bT_bB \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_bB \mid b \\ C \to T_cC \mid c \\ T_a \to a \\ T_b \to b \\ T_c \to c \end{cases}$$

5. *Finally, we introduce new variables, with binary productions, to simulate the productions of the form* $A \to w$, *where* $w \in V^*$ *and* $|w| \geq 2$, *and we obtain the equivalent grammar* G_4 :

Solution

$$G_3 : \begin{cases} S \to AB \mid T_aBC \mid SBS \mid T_aB \mid T_bT_bB \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_bB \mid b \\ C \to T_cC \mid c \\ T_a \to a \\ T_b \to b \\ T_c \to c \end{cases}$$

5. *Finally, we introduce new variables, with binary productions, to simulate the productions of the form $A \to w$, where $w \in V^*$ and $|w| \geq 2$, and we obtain the equivalent grammar G_4 :*

$$G_4 : \begin{cases} S \to AB \mid T_aT_1 \mid ST_2 \mid T_aB \mid T_bT_3 \mid b \\ \\ \\ \\ \\ \end{cases}$$

Solution

$$G_3 : \begin{cases} S \to AB \mid T_aBC \mid SBS \mid T_aB \mid T_bT_bB \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_bB \mid b \\ C \to T_cC \mid c \\ T_a \to a \\ T_b \to b \\ T_c \to c \end{cases}$$

5. *Finally, we introduce new variables, with binary productions, to simulate the productions of the form $A \to w$, where $w \in V^*$ and $|w| \geq 2$, and we obtain the equivalent grammar G_4 :*

$$G_4 : \begin{cases} S \to AB \mid T_aT_1 \mid ST_2 \mid T_aB \mid T_bT_3 \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \end{cases}$$

Solution

$$G_3 : \begin{cases} S \to AB \mid T_aBC \mid SBS \mid T_aB \mid T_bT_bB \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_bB \mid b \\ C \to T_cC \mid c \\ T_a \to a \\ T_b \to b \\ T_c \to c \end{cases}$$

5. *Finally, we introduce new variables, with binary productions, to simulate the productions of the form $A \to w$, where $w \in V^*$ and $|w| \geq 2$, and we obtain the equivalent grammar G_4 :*

$$G_4 : \begin{cases} S \to AB \mid T_aT_1 \mid ST_2 \mid T_aB \mid T_bT_3 \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_3 \mid b \end{cases}$$

Solution

$$G_3 : \begin{cases} S \to AB \mid T_aBC \mid SBS \mid T_aB \mid T_bT_bB \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_bB \mid b \\ C \to T_cC \mid c \\ T_a \to a \\ T_b \to b \\ T_c \to c \end{cases}$$

5. *Finally, we introduce new variables, with binary productions, to simulate the productions of the form $A \to w$, where $w \in V^*$ and $|w| \geq 2$, and we obtain the equivalent grammar G_4 :*

$$G_4 : \begin{cases} S \to AB \mid T_aT_1 \mid ST_2 \mid T_aB \mid T_bT_3 \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_3 \mid b \\ C \to T_cC \mid c \end{cases}$$

Solution

$$G_3 : \begin{cases} S \to AB \mid T_aBC \mid SBS \mid T_aB \mid T_bT_bB \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_bB \mid b \\ C \to T_cC \mid c \\ T_a \to a \\ T_b \to b \\ T_c \to c \end{cases}$$

5. *Finally, we introduce new variables, with binary productions, to simulate the productions of the form $A \to w$, where $w \in V^*$ and $|w| \geq 2$, and we obtain the equivalent grammar G_4 :*

$$G_4 : \begin{cases} S \to AB \mid T_aT_1 \mid ST_2 \mid T_aB \mid T_bT_3 \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_3 \mid b \\ C \to T_cC \mid c \\ T_1 \to BC \end{cases}$$

Solution

$$G_3 : \begin{cases} S \to AB \mid T_aBC \mid SBS \mid T_aB \mid T_bT_bB \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_bB \mid b \\ C \to T_cC \mid c \\ T_a \to a \\ T_b \to b \\ T_c \to c \end{cases}$$

5. *Finally, we introduce new variables, with binary productions, to simulate the productions of the form $A \to w$, where $w \in V^*$ and $|w| \geq 2$, and we obtain the equivalent grammar G_4 :*

$$G_4 : \begin{cases} S \to AB \mid T_aT_1 \mid ST_2 \mid T_aB \mid T_bT_3 \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_3 \mid b \\ C \to T_cC \mid c \\ T_1 \to BC \\ T_2 \to BS \end{cases}$$

Solution

$$G_3 : \begin{cases} S \to AB \mid T_aBC \mid SBS \mid T_aB \mid T_bT_bB \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_bB \mid b \\ C \to T_cC \mid c \\ T_a \to a \\ T_b \to b \\ T_c \to c \end{cases}$$

5. *Finally, we introduce new variables, with binary productions, to simulate the productions of the form $A \to w$, where $w \in V^*$ and $|w| \geq 2$, and we obtain the equivalent grammar G_4 :*

$$G_4 : \begin{cases} S \to AB \mid T_aT_1 \mid ST_2 \mid T_aB \mid T_bT_3 \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_3 \mid b \\ C \to T_cC \mid c \\ T_1 \to BC \\ T_2 \to BS \\ T_3 \to T_bB \end{cases}$$

Solution

$$G_3 : \begin{cases} S \to AB \mid T_aBC \mid SBS \mid T_aB \mid T_bT_bB \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_bB \mid b \\ C \to T_cC \mid c \\ T_a \to a \\ T_b \to b \\ T_c \to c \end{cases}$$

5. *Finally, we introduce new variables, with binary productions, to simulate the productions of the form $A \to w$, where $w \in V^*$ and $|w| \geq 2$, and we obtain the equivalent grammar G_4 :*

$$G_4 : \begin{cases} S \to AB \mid T_aT_1 \mid ST_2 \mid T_aB \mid T_bT_3 \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_3 \mid b \\ C \to T_cC \mid c \\ T_1 \to BC \\ T_2 \to BS \\ T_3 \to T_bB \\ T_a \to a \end{cases}$$

Solution

$$G_3 : \begin{cases} S \to AB \mid T_aBC \mid SBS \mid T_aB \mid T_bT_bB \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_bB \mid b \\ C \to T_cC \mid c \\ T_a \to a \\ T_b \to b \\ T_c \to c \end{cases}$$

5. *Finally, we introduce new variables, with binary productions, to simulate the productions of the form $A \to w$, where $w \in V^*$ and $|w| \geq 2$, and we obtain the equivalent grammar G_4 :*

$$G_4 : \begin{cases} S \to AB \mid T_aT_1 \mid ST_2 \mid T_aB \mid T_bT_3 \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_3 \mid b \\ C \to T_cC \mid c \\ T_1 \to BC \\ T_2 \to BS \\ T_3 \to T_bB \\ T_a \to a \\ T_b \to b \end{cases}$$

Solution

$$G_3 : \begin{cases} S \to AB \mid T_aBC \mid SBS \mid T_aB \mid T_bT_bB \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_bB \mid b \\ C \to T_cC \mid c \\ T_a \to a \\ T_b \to b \\ T_c \to c \end{cases}$$

5. *Finally, we introduce new variables, with binary productions, to simulate the productions of the form $A \to w$, where $w \in V^*$ and $|w| \geq 2$, and we obtain the equivalent grammar G_4 :*

$$G_4 : \begin{cases} S \to AB \mid T_aT_1 \mid ST_2 \mid T_aB \mid T_bT_3 \mid b \\ A \to T_aA \mid a \mid T_cC \mid c \\ B \to T_bT_3 \mid b \\ C \to T_cC \mid c \\ T_1 \to BC \\ T_2 \to BS \\ T_3 \to T_bB \\ T_a \to a \\ T_b \to b \\ T_c \to c \end{cases}$$

What additional requirement may be needed for the application of CNF?

What additional requirement may be needed for the application of CNF?

In some applications of CNF, it is necessary to require that the start variable S does not appear on the right-hand side of any production.

What additional requirement may be needed for the application of CNF?

In some applications of CNF, it is necessary to require that the start variable S does not appear on the right-hand side of any production.

What is S called if it appears on the right-hand side of a production in a CFG?

What additional requirement may be needed for the application of CNF?

In some applications of CNF, it is necessary to require that the start variable S does not appear on the right-hand side of any production.

What is S called if it appears on the right-hand side of a production in a CFG?

If S appears on the right-hand side of any production, it is said to be recursive since this leads to derivations of the form $S \overset{+}{\Longrightarrow} uSv$, with $u, v \in (V \cup \Sigma)$.

Is it possible to transform a CFG into an equivalent CFG where the start variable is not recursive?

Is it possible to transform a CFG into an equivalent CFG where the start variable is not recursive?

Theorem

Given a CFG $G = (V, \Sigma, S, P)$, we can construct an equivalent CFG $G' = (V', \Sigma, S', P')$ such that the start symbol S' of G' does not appear on the right-hand side of any productions in G'.

Is it possible to transform a CFG into an equivalent CFG where the start variable is not recursive?

Theorem

Given a CFG $G = (V, \Sigma, S, P)$, we can construct an equivalent CFG $G' = (V', \Sigma, S', P')$ such that the start symbol S' of G' does not appear on the right-hand side of any productions in G'.

But, how is the process to transform a CFG into an equivalent CFG where the start variable is not recursive?

Is it possible to transform a CFG into an equivalent CFG where the start variable is not recursive?

Theorem

Given a CFG $G = (V, \Sigma, S, P)$, we can construct an equivalent CFG $G' = (V', \Sigma, S', P')$ such that the start symbol S' of G' does not appear on the right-hand side of any productions in G'.

But, how is the process to transform a CFG into an equivalent CFG where the start variable is not recursive?

The new grammar G' must have one more variable, the variable S', which will act as the new start variable. That is, $V' = V \cup S'$. The set of productions P' will be given by $P' = P \cup S' \to S$. It is clear that $L(G) = L(G')$, and the start symbol S' will not appear on the right-hand side of any productions. As can be seen, the only role of S' will be to initiate derivations.

Example

Find an equivalent CFG G' for the following grammar G such that the start variable of G' is not recursive

Example

Find an equivalent CFG G' for the following grammar G such that the start variable of G' is not recursive

$$G : \begin{cases} S \to ASB \mid BB \\ A \to aA \mid a \\ B \to bBS \mid \lambda \end{cases}$$

Example

Find an equivalent CFG G' for the following grammar G such that the start variable of G' is not recursive

$$G : \begin{cases} S \to ASB \mid BB \\ A \to aA \mid a \\ B \to bBS \mid \lambda \end{cases}$$

Solution

As mentioned earlier, the only change is to add S' so that S' does not appear on the right-hand side of any production. This leads to:

Example

Find an equivalent CFG G' for the following grammar G such that the start variable of G' is not recursive

$$G : \begin{cases} S \to ASB \mid BB \\ A \to aA \mid a \\ B \to bBS \mid \lambda \end{cases}$$

Solution

As mentioned earlier, the only change is to add S' so that S' does not appear on the right-hand side of any production. This leads to:

$$G' : \begin{cases} S' \to S \\ \\ \\ \end{cases}$$

Example

Find an equivalent CFG G' for the following grammar G such that the start variable of G' is not recursive

$$G : \begin{cases} S \to ASB \mid BB \\ A \to aA \mid a \\ B \to bBS \mid \lambda \end{cases}$$

Solution

As mentioned earlier, the only change is to add S' so that S' does not appear on the right-hand side of any production. This leads to:

$$G' : \begin{cases} S' \to S \\ S \to ASB \mid BB \\ \\ \end{cases}$$

Example

Find an equivalent CFG G' for the following grammar G such that the start variable of G' is not recursive

$$G : \begin{cases} S \to ASB \mid BB \\ A \to aA \mid a \\ B \to bBS \mid \lambda \end{cases}$$

Solution

As mentioned earlier, the only change is to add S' so that S' does not appear on the right-hand side of any production. This leads to:

$$G' : \begin{cases} S' \to S \\ S \to ASB \mid BB \\ A \to aA \mid a \\ \end{cases}$$

Example

Find an equivalent CFG G' for the following grammar G such that the start variable of G' is not recursive

$$G : \begin{cases} S \to ASB \mid BB \\ A \to aA \mid a \\ B \to bBS \mid \lambda \end{cases}$$

Solution

As mentioned earlier, the only change is to add S' so that S' does not appear on the right-hand side of any production. This leads to:

$$G' : \begin{cases} S' \to S \\ S \to ASB \mid BB \\ A \to aA \mid a \\ B \to bBS \mid \lambda \end{cases}$$

When is a CFG G in Greibach Normal Form (GNF)?

Grammars and Languages

When is a CFG G in Greibach Normal Form (GNF)?

1. The initial variable is not recursive.

When is a CFG G in Greibach Normal Form (GNF)?

1. The initial variable is not recursive.
2. G has no useless variables.

Grammars and Languages

When is a CFG G in Greibach Normal Form (GNF)?

1. The initial variable is not recursive.
2. G has no useless variables.
3. G has no λ productions (except possibly $S \to \lambda$).

Grammars and Languages

When is a CFG G in Greibach Normal Form (GNF)?

1. The initial variable is not recursive.
2. G has no useless variables.
3. G has no λ productions (except possibly $S \to \lambda$).
4. All productions are of the form: $A \to a$ (simple productions) or $A \to aB_1B_2...B_k$, where B_i are variables.

Grammars and Languages

When is a CFG G in Greibach Normal Form (GNF)?

1. The initial variable is not recursive.
2. G has no useless variables.
3. G has no λ productions (except possibly $S \to \lambda$).
4. All productions are of the form: $A \to a$ (simple productions) or $A \to aB_1B_2...B_k$, where B_i are variables.

What are the notable characteristics of a grammar in GNF?

Grammars and Languages

When is a CFG G in Greibach Normal Form (GNF)?

1. The initial variable is not recursive.
2. G has no useless variables.
3. G has no λ productions (except possibly $S \to \lambda$).
4. All productions are of the form: $A \to a$ (simple productions) or $A \to aB_1B_2...B_k$, where B_i are variables.

What are the notable characteristics of a grammar in GNF?

1. A single terminal appears at each step.

Grammars and Languages

When is a CFG G in Greibach Normal Form (GNF)?

1. The initial variable is not recursive.
2. G has no useless variables.
3. G has no λ productions (except possibly $S \to \lambda$).
4. All productions are of the form: $A \to a$ (simple productions) or $A \to aB_1B_2...B_k$, where B_i are variables.

What are the notable characteristics of a grammar in GNF?

1. A single terminal appears at each step.
2. The derivation of a string of length n ($n \geq 1$) has exactly n steps.

Grammars and Languages

When is a CFG G in Greibach Normal Form (GNF)?

1. The initial variable is not recursive.
2. G has no useless variables.
3. G has no λ productions (except possibly $S \to \lambda$).
4. All productions are of the form: $A \to a$ (simple productions) or $A \to aB_1B_2...B_k$, where B_i are variables.

What are the notable characteristics of a grammar in GNF?

1. A single terminal appears at each step.
2. The derivation of a string of length n ($n \geq 1$) has exactly n steps.

Can a CFG G be converted to an equivalent grammar in GNF?

Grammars and Languages

When is a CFG G in Greibach Normal Form (GNF)?

1. The initial variable is not recursive.
2. G has no useless variables.
3. G has no λ productions (except possibly $S \to \lambda$).
4. All productions are of the form: $A \to a$ (simple productions) or $A \to aB_1B_2...B_k$, where B_i are variables.

What are the notable characteristics of a grammar in GNF?

1. A single terminal appears at each step.
2. The derivation of a string of length n ($n \geq 1$) has exactly n steps.

Can a CFG G be converted to an equivalent grammar in GNF?

There exists an algorithmic procedure to transform a given CFG into an equivalent grammar in GNF.

Grammars and Languages

Before presenting the algorithm, we must have certain concepts clear.

Before presenting the algorithm, we must have certain concepts clear.
What is a left-recursive variable?

Before presenting the algorithm, we must have certain concepts clear.
What is a left-recursive variable?

Definition

A variable is called left-recursive if it has a production of the form:

Before presenting the algorithm, we must have certain concepts clear.
What is a left-recursive variable?

Definition

A variable is called left-recursive if it has a production of the form:

$$A \to Aw,\ w \in (V \cup \Sigma)^*.$$

How can left recursion be eliminated?

How can left recursion be eliminated?

Theorem 1

The productions of any variable A can be divided into two classes:

How can left recursion be eliminated?

Theorem 1

The productions of any variable A can be divided into two classes:

$$\begin{cases} A \to A\alpha_1 \mid A\alpha_2 \mid \cdots \mid A\alpha_n \\ A \to \beta_1 \mid \beta_2 \mid \cdots \mid \beta_m \end{cases}$$

How can left recursion be eliminated?

Theorem 1

The productions of any variable A can be divided into two classes:

$$\begin{cases} A \to A\alpha_1 \mid A\alpha_2 \mid \cdots \mid A\alpha_n \\ A \to \beta_1 \mid \beta_2 \mid \cdots \mid \beta_m \end{cases}$$

where $\alpha_i, , \beta_i \in (V \cup \Sigma)^*$ *and the first symbol of* β_i *is different from A. Without altering the generated language, the above productions can be simulated by replacing them with the following:*

How can left recursion be eliminated?

Theorem 1

The productions of any variable A can be divided into two classes:

$$\begin{cases} A \to A\alpha_1 \mid A\alpha_2 \mid \cdots \mid A\alpha_n \\ A \to \beta_1 \mid \beta_2 \mid \cdots \mid \beta_m \end{cases}$$

where $\alpha_i, , \beta_i \in (V \cup \Sigma)^*$ *and the first symbol of* β_i *is different from A. Without altering the generated language, the above productions can be simulated by replacing them with the following:*

$$\begin{cases} A \to \beta_1 \mid \beta_2 \mid \cdots \mid \beta_m \mid \beta_1 Z \mid \beta_2 Z \mid \cdots \mid \beta_m Z \\ Z \to \alpha_1 \mid \alpha_2 \mid \cdots \mid \alpha_n \mid \alpha_1 Z \mid \alpha_2 Z \mid \cdots \mid \alpha_n Z \end{cases}$$

How can left recursion be eliminated?

Theorem 1

The productions of any variable A can be divided into two classes:

$$\begin{cases} A \to A\alpha_1 \mid A\alpha_2 \mid \cdots \mid A\alpha_n \\ A \to \beta_1 \mid \beta_2 \mid \cdots \mid \beta_m \end{cases}$$

where $\alpha_i, , \beta_i \in (V \cup \Sigma)^*$ *and the first symbol of* β_i *is different from A. Without altering the generated language, the above productions can be simulated by replacing them with the following:*

$$\begin{cases} A \to \beta_1 \mid \beta_2 \mid \cdots \mid \beta_m \mid \beta_1 Z \mid \beta_2 Z \mid \cdots \mid \beta_m Z \\ Z \to \alpha_1 \mid \alpha_2 \mid \cdots \mid \alpha_n \mid \alpha_1 Z \mid \alpha_2 Z \mid \cdots \mid \alpha_n Z \end{cases}$$

where Z is a completely new variable.

What follows from the previous theorem?

What follows from the previous theorem?

Lemma 1

In any CFG, a production $A \to uBv$ can be replaced (simulated) by:

$$A \to uw_1v \mid uw_2v \mid \cdots \mid uw_nv$$

where $B \to w_1, |, w_2, | \cdots, |, w_n$ are all the productions of B.

How is a CFG G converted to an equivalent grammar in GNF?

How is a CFG G converted to an equivalent grammar in GNF?

1. We assume that the grammar is in CNF.

How is a CFG G converted to an equivalent grammar in GNF?

1. We assume that the grammar is in CNF.
2. Enumerate the variables in an arbitrary but fixed order during the procedure. S must be the variable with order 1.

How is a CFG G converted to an equivalent grammar in GNF?

1. We assume that the grammar is in CNF.
2. Enumerate the variables in an arbitrary but fixed order during the procedure. S must be the variable with order 1.
3. For each variable A in the original grammar, following the chosen order, modify its productions so that the first symbol of the body of each production is a terminal or a variable with an order greater than A. To achieve this, use the left recursion elimination theorem (theorem 1) and lemma 1, as many times as necessary.

How is a CFG G converted to an equivalent grammar in GNF?

1. We assume that the grammar is in CNF.
2. Enumerate the variables in an arbitrary but fixed order during the procedure. S must be the variable with order 1.
3. For each variable A in the original grammar, following the chosen order, modify its productions so that the first symbol of the body of each production is a terminal or a variable with an order greater than A. To achieve this, use the left recursion elimination theorem (theorem 1) and lemma 1, as many times as necessary.
4. Use lemma 1 to modify the productions of the original variables so that the first symbol of the body of each production is a terminal. This is done following the reverse order of enumeration of the variables.

How is a CFG G converted to an equivalent grammar in GNF?

1. We assume that the grammar is in CNF.
2. Enumerate the variables in an arbitrary but fixed order during the procedure. S must be the variable with order 1.
3. For each variable A in the original grammar, following the chosen order, modify its productions so that the first symbol of the body of each production is a terminal or a variable with an order greater than A. To achieve this, use the left recursion elimination theorem (theorem 1) and lemma 1, as many times as necessary.
4. Use lemma 1 to modify the productions of the original variables so that the first symbol of the body of each production is a terminal. This is done following the reverse order of enumeration of the variables.
5. Again, use lemma 1 to modify the productions of the new variables so that the first symbol of the body of each production is a terminal.

Example

Find a grammar in GNF equivalent to the following grammar (which is in CNF):

Example

Find a grammar in GNF equivalent to the following grammar (which is in CNF):

$$G : \begin{cases} S \to AA \mid a \\ A \to AA \mid b \end{cases}$$

Example

Find a grammar in GNF equivalent to the following grammar (which is in CNF):

$$G : \begin{cases} S \to AA \mid a \\ A \to AA \mid b \end{cases}$$

Solution

Step 1: *There is only one possible order for the variables:* S, A.

Example

Find a grammar in GNF equivalent to the following grammar (which is in CNF):

$$G : \begin{cases} S \to AA \mid a \\ A \to AA \mid b \end{cases}$$

Solution

Step 1: *There is only one possible order for the variables: S, A.*

Step 2: *In this step, we just need to eliminate left recursion from the variable A. By doing so, we obtain the grammar:*

Example

Find a grammar in GNF equivalent to the following grammar (which is in CNF):

$$G : \begin{cases} S \to AA \mid a \\ A \to AA \mid b \end{cases}$$

Solution

Step 1: *There is only one possible order for the variables: S, A.*

Step 2: *In this step, we just need to eliminate left recursion from the variable A. By doing so, we obtain the grammar:*

$$\begin{cases} S \to AA \mid a \\ A \to b \mid bZ \\ Z \to A \mid AZ \end{cases}$$

Solution

$$\begin{cases} S \to AA \mid a \\ A \to b \mid bZ \\ Z \to A \mid AZ \end{cases}$$

Solution

$$\begin{cases} S \to AA \mid a \\ A \to b \mid bZ \\ Z \to A \mid AZ \end{cases}$$

Step 3: *Modify the productions of the original variables so that the first symbol of the body of each production is a terminal:*

Solution

$$\begin{cases} S \to AA \mid a \\ A \to b \mid bZ \\ Z \to A \mid AZ \end{cases}$$

Step 3: *Modify the productions of the original variables so that the first symbol of the body of each production is a terminal:*

$$\begin{cases} S \to bA \mid bZA \mid a \\ A \to b \mid bZ \\ Z \to A \mid AZ \end{cases}$$

Solution

$$\begin{cases} S \to AA \mid a \\ A \to b \mid bZ \\ Z \to A \mid AZ \end{cases}$$

Step 3: *Modify the productions of the original variables so that the first symbol of the body of each production is a terminal:*

$$\begin{cases} S \to bA \mid bZA \mid a \\ A \to b \mid bZ \\ Z \to A \mid AZ \end{cases}$$

Step 4: *Modify the productions of the new variables so that the first symbol of the body of each production is a terminal:*

Solution

$$\begin{cases} S \to AA \mid a \\ A \to b \mid bZ \\ Z \to A \mid AZ \end{cases}$$

Step 3: *Modify the productions of the original variables so that the first symbol of the body of each production is a terminal:*

$$\begin{cases} S \to bA \mid bZA \mid a \\ A \to b \mid bZ \\ Z \to A \mid AZ \end{cases}$$

Step 4: *Modify the productions of the new variables so that the first symbol of the body of each production is a terminal:*

$$\begin{cases} S \to bA \mid bZA \mid a \\ A \to b \mid bZ \\ Z \to b \mid bZ \mid bZZ \end{cases}$$

Example

Find a grammar in GNF equivalent to the following grammar (which is in CNF):

Example

Find a grammar in GNF equivalent to the following grammar (which is in CNF):

$$G : \begin{cases} S \to AB \mid BC \\ A \to AB \mid a \\ B \to AA \mid CB \mid a \\ C \to a \mid b \end{cases}$$

Example

Find a grammar in GNF equivalent to the following grammar (which is in CNF):

$$G : \begin{cases} S \to AB \mid BC \\ A \to AB \mid a \\ B \to AA \mid CB \mid a \\ C \to a \mid b \end{cases}$$

Solution

Step 1: *Variable order: S, B, A, C. This order is very suitable because the body of the productions of B starts with A or C, which are variables of higher order.*

Example

Find a grammar in GNF equivalent to the following grammar (which is in CNF):

$$G: \begin{cases} S \to AB \mid BC \\ A \to AB \mid a \\ B \to AA \mid CB \mid a \\ C \to a \mid b \end{cases}$$

Solution

Step 1: *Variable order: S, B, A, C. This order is very suitable because the body of the productions of B starts with A or C, which are variables of higher order.*

$$\begin{cases} S \to AB \mid BC \\ B \to AA \mid CB \mid a \\ A \to AB \mid a \\ C \to a \mid b \end{cases}$$

Solution

$$\begin{cases} S \rightarrow AB \mid BC \\ B \rightarrow AA \mid CB \mid a \\ A \rightarrow AB \mid a \\ C \rightarrow a \mid b \end{cases}$$

Solution

$$\begin{cases} S \rightarrow AB \mid BC \\ B \rightarrow AA \mid CB \mid a \\ A \rightarrow AB \mid a \\ C \rightarrow a \mid b \end{cases}$$

Step 2: *In this step, we just need to eliminate left recursion from the variable A. By doing so, we obtain the grammar:*

Solution

$$\begin{cases} S \rightarrow AB \mid BC \\ B \rightarrow AA \mid CB \mid a \\ A \rightarrow AB \mid a \\ C \rightarrow a \mid b \end{cases}$$

Step 2: *In this step, we just need to eliminate left recursion from the variable A. By doing so, we obtain the grammar:*

$$\begin{cases} S \rightarrow AB \mid BC \\ B \rightarrow AA \mid CB \mid a \\ A \rightarrow a \mid aZ \\ C \rightarrow a \mid b \\ Z \rightarrow B \mid BZ \end{cases}$$

Solution

$$\begin{cases} S \rightarrow AB \mid BC \\ B \rightarrow AA \mid CB \mid a \\ A \rightarrow AB \mid a \\ C \rightarrow a \mid b \end{cases}$$

Step 2: *In this step, we just need to eliminate left recursion from the variable A. By doing so, we obtain the grammar:*

$$\begin{cases} S \rightarrow AB \mid BC \\ B \rightarrow AA \mid CB \mid a \\ A \rightarrow a \mid aZ \\ C \rightarrow a \mid b \\ Z \rightarrow B \mid BZ \end{cases}$$

Step 3: *Modify the productions of the original variables so that the first symbol of the body of each production is a terminal:*

Solution

$$\begin{cases} S \rightarrow AB \mid BC \\ B \rightarrow AA \mid CB \mid a \\ A \rightarrow AB \mid a \\ C \rightarrow a \mid b \end{cases}$$

Step 2: *In this step, we just need to eliminate left recursion from the variable A. By doing so, we obtain the grammar:*

$$\begin{cases} S \rightarrow AB \mid BC \\ B \rightarrow AA \mid CB \mid a \\ A \rightarrow a \mid aZ \\ C \rightarrow a \mid b \\ Z \rightarrow B \mid BZ \end{cases}$$

Step 3: *Modify the productions of the original variables so that the first symbol of the body of each production is a terminal:*

$$\begin{cases} S \rightarrow aB \mid aZB \mid aAC \mid aZAC \mid aBC \mid bBC \mid aC \\ B \rightarrow aA \mid aZA \mid aB \mid bB \mid a \\ A \rightarrow a \mid aZ \\ C \rightarrow a \mid b \\ Z \rightarrow B \mid BZ \end{cases}$$

Solution

$$\begin{cases} S \to aB \mid aZB \mid aAC \mid aZAC \mid aBC \mid bBC \mid aC \\ B \to aA \mid aZA \mid aB \mid bB \mid a \\ A \to a \mid aZ \\ C \to a \mid b \\ Z \to B \mid BZ \end{cases}$$

Solution

$$\begin{cases} S \to aB \mid aZB \mid aAC \mid aZAC \mid aBC \mid bBC \mid aC \\ B \to aA \mid aZA \mid aB \mid bB \mid a \\ A \to a \mid aZ \\ C \to a \mid b \\ Z \to B \mid BZ \end{cases}$$

Step 4: *Modify the productions of the new variables so that the first symbol of the body of each production is a terminal:*

Solution

$$\begin{cases} S \to aB \mid aZB \mid aAC \mid aZAC \mid aBC \mid bBC \mid aC \\ B \to aA \mid aZA \mid aB \mid bB \mid a \\ A \to a \mid aZ \\ C \to a \mid b \\ Z \to B \mid BZ \end{cases}$$

Step 4: *Modify the productions of the new variables so that the first symbol of the body of each production is a terminal:*

$$\begin{cases} S \to aB \mid aZB \mid aAC \mid aZAC \mid aBC \mid bBC \mid aC \\ B \to aA \mid aZA \mid aB \mid bB \mid a \\ A \to a \mid aZ \\ C \to a \mid b \\ Z \to aA \mid aZA \mid aB \mid bB \mid a \mid aAZ \mid aZAZ \mid aBZ \mid bBZ \mid aZ \end{cases}$$

In what does a decision problem for CFGs consist of?

In what does a decision problem for CFGs consist of?

Given a property $\mathcal{P}$, regarding context-free grammars, a decision problem for $\mathcal{P}$ consists of seeking an algorithm, applicable to an arbitrary CFG G, that answers YES or NO to the question: Does G satisfy property $\mathcal{P}$?

In what does a decision problem for CFGs consist of?

Given a property $\mathcal{P}$, regarding context-free grammars, a decision problem for $\mathcal{P}$ consists of seeking an algorithm, applicable to an arbitrary CFG G, that answers YES or NO to the question: Does G satisfy property $\mathcal{P}$?

What could be useful for solving more complex decision algorithms?

In what does a decision problem for CFGs consist of?

Given a property $\mathcal{P}$, regarding context-free grammars, a decision problem for $\mathcal{P}$ consists of seeking an algorithm, applicable to an arbitrary CFG G, that answers YES or NO to the question: Does G satisfy property $\mathcal{P}$?

What could be useful for solving more complex decision algorithms?

The algorithms recently seen (for finding nullable, reachable, and generating variables, etc.) are often useful in designing more complex decision algorithms.

Grammars and Languages

How can we solve the emptiness problem for CFGs? That is, determining if given a grammar $G = (V, \Sigma, S, P)$, $L(G) \neq \emptyset$?

Grammars and Languages

How can we solve the emptiness problem for CFGs? That is, determining if given a grammar $G = (V, \Sigma, S, P)$, $L(G) \neq \emptyset$?

The decision algorithm would be: execute the algorithm to determine the set **GEN** of generating variables. $L(G) \neq \emptyset$ if and only if $S \in$ **GEN**.

Grammars and Languages

How can we solve the emptiness problem for CFGs? That is, determining if given a grammar $G = (V, \Sigma, S, P)$, $L(G) \neq \emptyset$?

The decision algorithm would be: execute the algorithm to determine the set **GEN** of generating variables. $L(G) \neq \emptyset$ if and only if $S \in$ **GEN**.

How is the membership problem solved for CFGs? That is, determining if given a grammar $G = (V, \Sigma, S, P)$ and a string $w \in \Sigma^*$, $w \in L(G)$?

How can we solve the emptiness problem for CFGs? That is, determining if given a grammar $G = (V, \Sigma, S, P)$, $L(G) \neq \emptyset$?

The decision algorithm would be: execute the algorithm to determine the set **GEN** of generating variables. $L(G) \neq \emptyset$ if and only if $S \in$ **GEN**.

How is the membership problem solved for CFGs? That is, determining if given a grammar $G = (V, \Sigma, S, P)$ and a string $w \in \Sigma^*$, $w \in L(G)$?

- To solve this problem, first convert G to CNF, with non-recursive start variable.

How can we solve the emptiness problem for CFGs? That is, determining if given a grammar $G = (V, \Sigma, S, P)$, $L(G) \neq \emptyset$?

The decision algorithm would be: execute the algorithm to determine the set **GEN** of generating variables. $L(G) \neq \emptyset$ if and only if $S \in$ **GEN**.

How is the membership problem solved for CFGs? That is, determining if given a grammar $G = (V, \Sigma, S, P)$ and a string $w \in \Sigma^*$, $w \in L(G)$?

- To solve this problem, first convert G to CNF, with non-recursive start variable.
- From a CFG G in CNF, a rather inefficient algorithm can be designed to decide if $w \in L(G)$.

How can we solve the emptiness problem for CFGs? That is, determining if given a grammar $G = (V, \Sigma, S, P)$, $L(G) \neq \emptyset$?

The decision algorithm would be: execute the algorithm to determine the set **GEN** of generating variables. $L(G) \neq \emptyset$ if and only if $S \in$ **GEN**.

How is the membership problem solved for CFGs? That is, determining if given a grammar $G = (V, \Sigma, S, P)$ and a string $w \in \Sigma^*$, $w \in L(G)$?

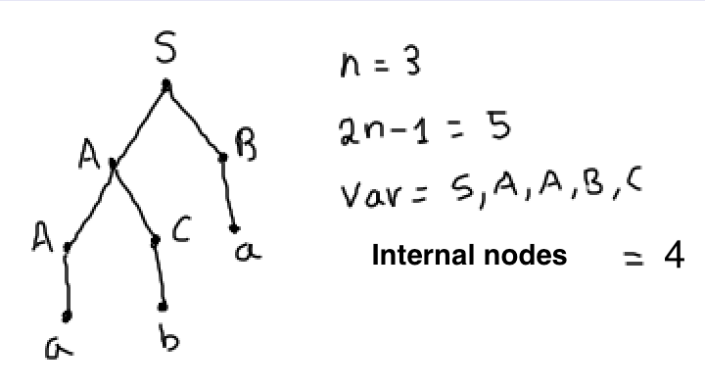
- To solve this problem, first convert G to CNF, with non-recursive start variable.
- From a CFG G in CNF, a rather inefficient algorithm can be designed to decide if $w \in L(G)$.
- Find all possible leftmost derivations (or derivation trees) that generate strings of length $n = |w|$.

Grammars and Languages

- A derivation tree for a string of length n has exactly $2n - 1$ nodes labeled with variables.

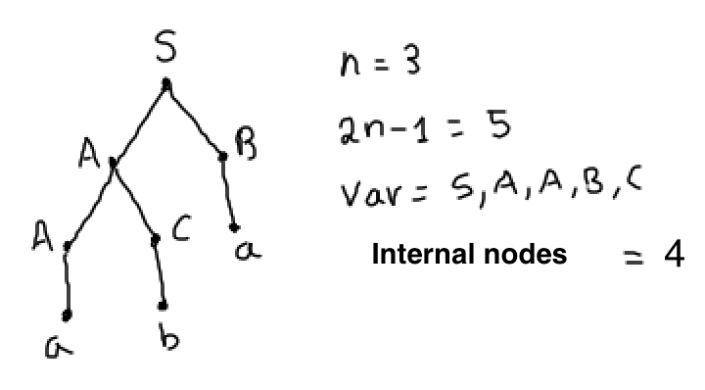
Grammars and Languages

- A derivation tree for a string of length n has exactly $2n - 1$ nodes labeled with variables.



Grammars and Languages

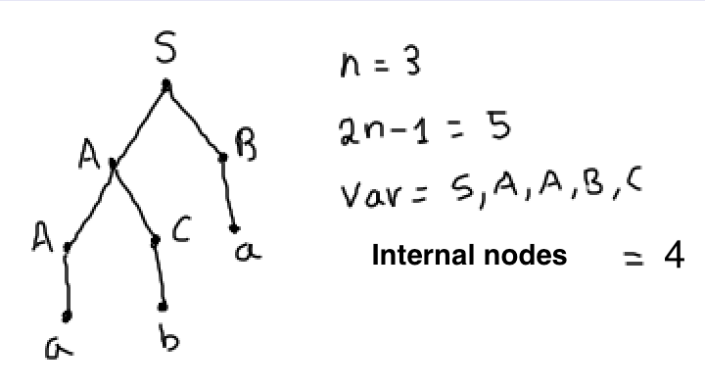
- A derivation tree for a string of length n has exactly $2n - 1$ nodes labeled with variables.



Since the root of the tree is S and S is not recursive, in a derivation tree for a string of length n, there are exactly $2n - 2$ interior nodes labeled with variables.

Grammars and Languages

- A derivation tree for a string of length n has exactly $2n - 1$ nodes labeled with variables.

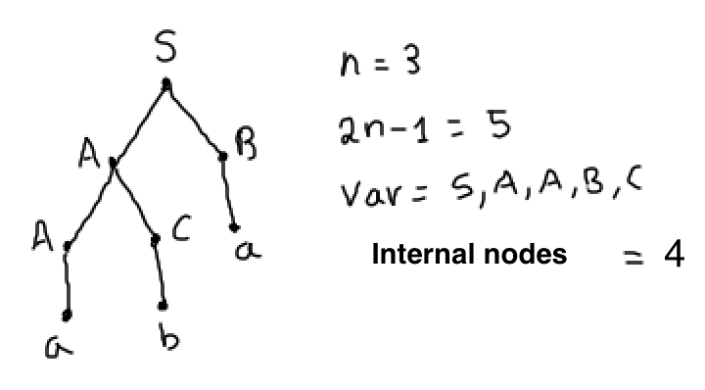


  Since the root of the tree is S and S is not recursive, in a derivation tree for a string of length n, there are exactly $2n - 2$ interior nodes labeled with variables.

- To determine whether a given string of length n is generated by G or not, all possible derivation trees with $2n - 2$ interior variables must be considered.

Grammars and Languages

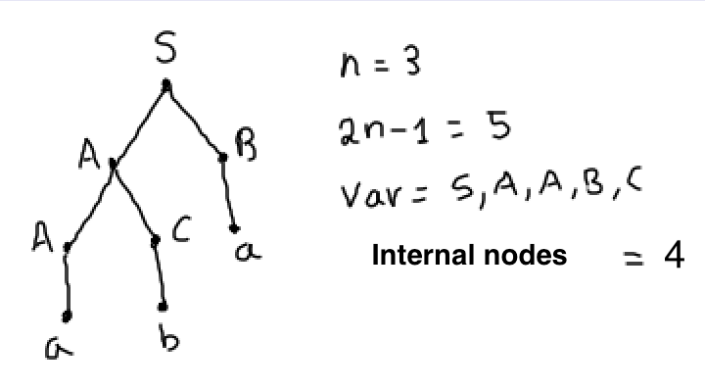
- A derivation tree for a string of length n has exactly $2n - 1$ nodes labeled with variables.



  Since the root of the tree is S and S is not recursive, in a derivation tree for a string of length n, there are exactly $2n - 2$ interior nodes labeled with variables.

- To determine whether a given string of length n is generated by G or not, all possible derivation trees with $2n - 2$ interior variables must be considered.
- This algorithm is inefficient because if G has k variables, at least k^{2n-2} trees must be checked (not counting possibilities for the leaves).

- A derivation tree for a string of length n has exactly $2n - 1$ nodes labeled with variables.



  Since the root of the tree is S and S is not recursive, in a derivation tree for a string of length n, there are exactly $2n - 2$ interior nodes labeled with variables.

- To determine whether a given string of length n is generated by G or not, all possible derivation trees with $2n - 2$ interior variables must be considered.
- This algorithm is inefficient because if G has k variables, at least k^{2n-2} trees must be checked (not counting possibilities for the leaves).
- Therefore, the procedure has exponential complexity with respect to the input size.

Is there any efficient algorithm to solve this problem?

Is there any efficient algorithm to solve this problem?

- Yes, there is a very efficient algorithm to solve the membership problem (its complexity is polynomial).

Is there any efficient algorithm to solve this problem?

- Yes, there is a very efficient algorithm to solve the membership problem (its complexity is polynomial).
- It uses a technique called dynamic programming (a technique for filling tables progressively, reusing previously obtained information).

Is there any efficient algorithm to solve this problem?

- Yes, there is a very efficient algorithm to solve the membership problem (its complexity is polynomial).
- It uses a technique called dynamic programming (a technique for filling tables progressively, reusing previously obtained information).
- This algorithm is called the CYK algorithm (named after the researchers Cocke, Younger, and Kasami).

What is the CYK algorithm?

What is the CYK algorithm?

- This algorithm takes as input a CFG G in CNF and a string of n terminals $w = a_1 a_2 \cdots a_n$.

What is the CYK algorithm?

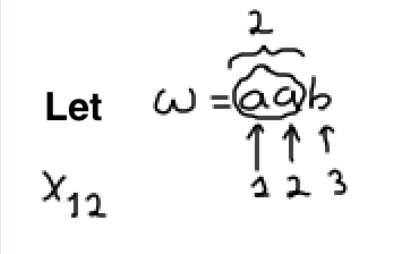
- This algorithm takes as input a CFG G in CNF and a string of n terminals $w = a_1 a_2 \cdots a_n$.
- It is applied by filling a table with n rows (one for each terminal of the input w) and n columns.

What is the CYK algorithm?

- This algorithm takes as input a CFG G in CNF and a string of n terminals $w = a_1 a_2 \cdots a_n$.
- It is applied by filling a table with n rows (one for each terminal of the input w) and n columns.
- X_{ij} is the set of variables from which the substring of w whose first symbol is at position i and whose length is j can be derived.

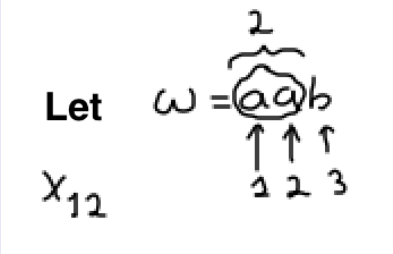
What is the CYK algorithm?

- This algorithm takes as input a CFG G in CNF and a string of n terminals $w = a_1 a_2 \cdots a_n$.
- It is applied by filling a table with n rows (one for each terminal of the input w) and n columns.
- X_{ij} is the set of variables from which the substring of w whose first symbol is at position i and whose length is j can be derived.



What is the CYK algorithm?

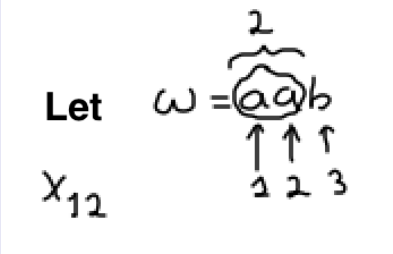
- This algorithm takes as input a CFG G in CNF and a string of n terminals $w = a_1 a_2 \cdots a_n$.
- It is applied by filling a table with n rows (one for each terminal of the input w) and n columns.
- X_{ij} is the set of variables from which the substring of w whose first symbol is at position i and whose length is j can be derived.



- That is, X_{ij} = set of variables A such that $A \overset{+}{\Longrightarrow} a_i a_{i+1} \cdots a_{i+j-1}$.

What is the CYK algorithm?

- This algorithm takes as input a CFG G in CNF and a string of n terminals $w = a_1 a_2 \cdots a_n$.
- It is applied by filling a table with n rows (one for each terminal of the input w) and n columns.
- X_{ij} is the set of variables from which the substring of w whose first symbol is at position i and whose length is j can be derived.



- That is, X_{ij} = set of variables A such that $A \overset{+}{\Longrightarrow} a_i a_{i+1} \cdots a_{i+j-1}$.
- By determining the sets X_{ij}, the possible ways to derive substrings of w that allow building a derivation of the complete string w are obtained.

- The table is filled column by column, from top to bottom; the first column ($j = 1$) corresponds to substrings of length 1, the second column ($j = 2$) corresponds to substrings of length 2, and so on. The last column ($j = n$) corresponds to the only substring of length n in w, which is the string w itself.

- The table is filled column by column, from top to bottom; the first column ($j = 1$) corresponds to substrings of length 1, the second column ($j = 2$) corresponds to substrings of length 2, and so on. The last column ($j = n$) corresponds to the only substring of length n in w, which is the string w itself.
- Variables that derive terminals at different positions in the string are found (first column).

- The table is filled column by column, from top to bottom; the first column ($j = 1$) corresponds to substrings of length 1, the second column ($j = 2$) corresponds to substrings of length 2, and so on. The last column ($j = n$) corresponds to the only substring of length n in w, which is the string w itself.
- Variables that derive terminals at different positions in the string are found (first column).
- Since j represents length, when $j = 1$, we find the variables that generate strings of length 1, and those are terminals.

- The table is filled column by column, from top to bottom; the first column ($j = 1$) corresponds to substrings of length 1, the second column ($j = 2$) corresponds to substrings of length 2, and so on. The last column ($j = n$) corresponds to the only substring of length n in w, which is the string w itself.
- Variables that derive terminals at different positions in the string are found (first column).
- Since j represents length, when $j = 1$, we find the variables that generate strings of length 1, and those are terminals.
- It follows that $w \in L(G)$ if and only if $S \in X_{1n}$.

Example

$$G: \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases} \qquad w = a\,a\,b$$

Example

$$G: \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases} \qquad w = a\,a\,b$$

$X_{11} = \{A\}$

$$w = \overbrace{a}^{1}\,a\,b$$

$$\begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

Example

$$G: \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases} \qquad w = a\,a\,b$$

$X_{11} = \{A\}$

$$w = \overbrace{a}^{1}\,a\,b$$

$$\begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

$X_{21} = \{A\}$

$$w = a\,\overbrace{a}^{1}\,b$$

$$\begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

Example

$$G: \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases} \qquad w = a\,a\,b$$

$X_{11} = \{A\}$

$$w = \overbrace{a}^{1}\ \underset{\substack{\uparrow \\ 1}}{a}\ \underset{\substack{\uparrow \\ 2}}{a}\ \underset{\substack{\uparrow \\ 3}}{b}$$

$X_{21} = \{A\}$

$$w = \underset{\substack{\uparrow \\ 1}}{a}\ \overbrace{a}^{1}\ \underset{\substack{\uparrow \\ 2}}{a}\ \underset{\substack{\uparrow \\ 3}}{b}$$

$X_{31} = \{B\}$

$$w = \underset{\substack{\uparrow \\ 1}}{a}\ \underset{\substack{\uparrow \\ 2}}{a}\ \overbrace{b}^{1}$$

Example

$$G: \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases} \qquad w = a\,a\,b$$

$X_{11} = \{A\}$

$$w = \overbrace{a}^{1}\ \underset{\substack{\uparrow \\ 1}}{}\ \underset{\substack{\uparrow \\ 2}}{a}\ \underset{\substack{\uparrow \\ 3}}{b}$$

$X_{12} = \{A\}$

$$w = \overbrace{a\ a}^{2}\ \underset{\substack{\uparrow \\ 3}}{b}$$

AA

$X_{21} = \{A\}$

$$w = a\ \overbrace{a}^{1}\ b$$

$X_{31} = \{B\}$

$$w = a\ a\ \overbrace{b}^{1}$$

Example

$$G: \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases} \qquad w = a\,a\,b$$

$X_{11} = \{A\}$

$$w = \overbrace{a}^{1}\ a\ b \qquad \begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

$X_{12} = \{A\}$

$$w = \overbrace{a\ a}^{2}\ b \qquad \begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

AA

$X_{21} = \{A\}$

$$w = a\ \overbrace{a}^{1}\ b \qquad \begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

$X_{22} = \{S\}$

$$w = a\ \overbrace{a\ b}^{2} \qquad \begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

AB

$X_{31} = \{B\}$

$$w = a\ a\ \overbrace{b}^{1} \qquad \begin{array}{ccc} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{array}$$

Example

$$G: \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases} \qquad w = a\,a\,b$$

$X_{11} = \{A\}$

$w = \overbrace{a}^{1}\,a\,b$ (positions 1 2 3)

$X_{12} = \{A\}$

$w = \overbrace{a\,a}^{2}\,b$ (positions 1 2 3)

$X_{13} = \{S\}$

$w = \overbrace{a\,a\,b}^{3}$ (positions 1 2 3)

$X_{21} = \{A\}$

$w = a\,\overbrace{a}^{1}\,b$ (positions 1 2 3)

$X_{22} = \{S\}$

$w = a\,\overbrace{a\,b}^{2}$ (positions 1 2 3)

$X_{31} = \{B\}$

$w = a\,a\,\overbrace{b}^{1}$ (positions 1 2 3)

AA

AB

AS

AB

What is the pseudocode for the CYK algorithm?

What is the pseudocode for the CYK algorithm?

Algoritmo CYK

INPUT:
Gramámar G in CNF and a string of n terminals $w = a_1 a_2 \cdots a_n$.

INITIALIZE:
$j = 1$. For each i, $1 \leq i \leq n$,
$X_{ij} = X_{i1}$:= set of variables A such that $A \to a_i$

REPEAT:
$j := j + 1$. For each i, $1 \leq i \leq n - j + 1$,
X_{ij} := set of variables A such that $A \to BC$ es
a production of G, with $B \in X_{ik}$ y $C \in X_{i+k,j-k}$,
taking into account all k such that $1 \leq k \leq j - 1$.

UNTIL: $j = n$.

OUTPUT: $w \in L(G)$ if and only if $S \in X_{1n}$.

Example

Let G be the grammar

$$G : \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases}$$

Example

Let G be the grammar

$$G : \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases}$$

and the string $w = aab$. *Use the CYK algorithm to determine if* $w \in L(G)$ *or not.*

Solution

		$j=1$	$j=2$	$j=3$
a	$i=1$			
a	$i=2$			
b	$i=3$			

$$G: \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases}$$

Solution

		$j=1$	$j=2$	$j=3$
a	$i=1$	$\{A\}$		
a	$i=2$	$\{A\}$		
b	$i=3$	$\{B\}$		

$X_{11} = \{A\}$

$X_{21} = \{A\}$

$X_{31} = \{B\}$

$$G: \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases}$$

Solution

		$j=1$	$j=2$	$j=3$
a	$i=1$	$\{A\}$	$\{A\}$	
a	$i=2$	$\{A\}$	$\{S\}$	
b	$i=3$	$\{B\}$		

$$G: \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases}$$

$X_{11} = \{A\}$

$X_{21} = \{A\}$

$X_{31} = \{B\}$

$X_{ij} \longrightarrow X_{ik} X_{i+k\, j-k}$

$j=2 \quad 1 \leq k \leq 1$

$i=1$

$X_{12} \to X_{11} X_{21}$

$X_{12} \to \{A\}\{A\} = \{AA\}$

$X_{12} = \{A\}$

$i=2$

$X_{22} \to X_{21} X_{31}$

$X_{22} \to \{A\}\{B\} = \{AB\}$

$X_{22} = \{S\}$

Solution

$$G: \begin{cases} S \to AB \\ A \to AA \mid a \\ B \to b \end{cases}$$

		$j=1$	$j=2$	$j=3$
a	$i=1$	$\{A\}$	$\{A\}$	$\{S\}$
a	$i=2$	$\{A\}$	$\{S\}$	
b	$i=3$	$\{B\}$		

$X_{11} = \{A\}$

$X_{21} = \{A\}$

$X_{31} = \{B\}$

$X_{ij} \longrightarrow X_{ik} X_{i+k\, j-k}$

$j=2 \quad 1 \leq k \leq 1$

$i=1$

$X_{12} \to X_{11} X_{21}$

$X_{12} \to \{A\}\{A\} = \{AA\}$

$X_{12} = \{A\}$

$i=2$

$X_{22} \to X_{21} X_{31}$

$X_{22} \to \{A\}\{B\} = \{AB\}$

$X_{22} = \{S\}$

$j=3 \quad 1 \leq k \leq 2$

$i=1$

$X_{13} \to X_{11} X_{22} \cup X_{12} X_{31}$

$X_{13} \to \{A\}\{S\} \cup \{A\}\{B\} = \{AS, AB\}$

$X_{13} = \{S\}$

$S \in X_{13}$

Example

Let G be the grammar

$$G : \begin{cases} S \to BA \mid AC \\ A \to CC \mid b \\ B \to AB \mid a \\ C \to BA \mid a \end{cases}$$

Example

Let G be the grammar

$$G : \begin{cases} S \to BA \mid AC \\ A \to CC \mid b \\ B \to AB \mid a \\ C \to BA \mid a \end{cases}$$

and the string $w = bbab$. Use the CYK algorithm to determine if $w \in L(G)$ or not.

Solution

		$j=1$	$j=2$	$j=3$	$j=4$
b	$i=1$				
b	$i=2$				
a	$i=3$				
b	$i=4$				

$$G: \begin{cases} S \to BA \mid AC \\ A \to CC \mid b \\ B \to AB \mid a \\ C \to BA \mid a \end{cases}$$

Solution

		$j=1$	$j=2$	$j=3$	$j=4$
b	$i=1$	$\{A\}$			
b	$i=2$	$\{A\}$			
a	$i=3$	$\{B,C\}$			
b	$i=4$	$\{A\}$			

$x_{11} = \{A\}$
$x_{21} = \{A\}$
$x_{31} = \{B,C\}$
$x_{41} = \{A\}$

$$G: \begin{cases} S \to BA \mid AC \\ A \to CC \mid b \\ B \to AB \mid a \\ C \to BA \mid a \end{cases}$$

Solution

		$j=1$	$j=2$	$j=3$	$j=4$
b	$i=1$	$\{A\}$	$\{\}$		
b	$i=2$	$\{A\}$	$\{B,S\}$		
a	$i=3$	$\{B,C\}$	$\{S,C\}$		
b	$i=4$	$\{A\}$			

$$G: \begin{cases} S \to BA \mid AC \\ A \to CC \mid b \\ B \to AB \mid a \\ C \to BA \mid a \end{cases}$$

$X_{11} = \{A\}$
$X_{21} = \{A\}$
$X_{31} = \{B,C\}$
$X_{41} = \{A\}$

$$X_{ij} \longrightarrow X_{ik} X_{i+k\, j-k}$$

$j=2 \quad 1 \leq k \leq 1$

$i=1$

$X_{12} \to X_{11} X_{21}$
$X_{12} \to \{A\}\{A\} = \{AA\}$
$X_{12} = \{\}$

$i=2$

$X_{22} \to X_{21} X_{31}$
$X_{22} \to \{A\}\{B,C\} = \{AB, AC\}$
$X_{22} = \{B,S\}$

$i=3$

$X_{32} \to X_{31} X_{41}$
$X_{32} \to \{B,C\}\{A\} = \{BA, CA\}$
$X_{32} = \{S,C\}$

Solution

		$j=1$	$j=2$	$j=3$	$j=4$
b	$i=1$	$\{A\}$	$\{\}$	$\{B\}$	
b	$i=2$	$\{A\}$	$\{B,S\}$	$\{S,C\}$	
a	$i=3$	$\{B,C\}$	$\{S,C\}$		
b	$i=4$	$\{A\}$			

$$G: \begin{cases} S \to BA \mid AC \\ A \to CC \mid b \\ B \to AB \mid a \\ C \to BA \mid a \end{cases}$$

$X_{11} = \{A\}$
$X_{21} = \{A\}$
$X_{31} = \{B,C\}$
$X_{41} = \{A\}$

$X_{ij} \longrightarrow X_{ik} X_{i+k\, j-k}$

$j=2 \quad 1 \leq k \leq 1$

$i=1$

$X_{12} \to X_{11} X_{21}$
$X_{12} \to \{A\}\{A\} = \{AA\}$
$X_{12} = \{\}$

$i=2$

$X_{22} \to X_{21} X_{31}$
$X_{22} \to \{A\}\{B,C\} = \{AB, AC\}$
$X_{22} = \{B,S\}$

$i=3$

$X_{32} \to X_{31} X_{41}$
$X_{32} \to \{B,C\}\{A\} = \{BA, CA\}$
$X_{32} = \{S,C\}$

$j=3 \quad 1 \leq k \leq 2$

$i=1$

$X_{13} \to X_{11} X_{22} \cup X_{12} X_{31}$
$X_{13} \to \{A\}\{B,S\} \cup \{\}\{B,C\} = \{AB, AS\}$
$X_{13} = \{B\}$

$i=2$

$X_{23} \to X_{21} X_{32} \cup X_{22} X_{41}$
$X_{23} \to \{A\}\{S,C\} \cup \{B,S\}\{A\} = \{AS, AC, BA, SA\}$
$X_{23} = \{S,C\}$

Solution

		$j=1$	$j=2$	$j=3$	$j=4$
b	$i=1$	$\{A\}$	$\{\}$	$\{B\}$	$\{S,C\}$
b	$i=2$	$\{A\}$	$\{B,S\}$	$\{S,C\}$	
a	$i=3$	$\{B,C\}$	$\{S,C\}$		
b	$i=4$	$\{A\}$			

$$G: \begin{cases} S \to BA \mid AC \\ A \to CC \mid b \\ B \to AB \mid a \\ C \to BA \mid a \end{cases}$$

$X_{11} = \{A\}$
$X_{21} = \{A\}$
$X_{31} = \{B,C\}$
$X_{41} = \{A\}$

$X_{ij} \longrightarrow X_{ik} X_{i+k\, j-k}$

$j=2 \quad 1 \le k \le 1$

$i=1$

$X_{12} \to X_{11} X_{21}$
$X_{12} \to \{A\}\{A\} = \{AA\}$
$X_{12} = \{\}$

$i=2$

$X_{22} \to X_{21} X_{31}$
$X_{22} \to \{A\}\{B,C\} = \{AB, AC\}$
$X_{22} = \{B,S\}$

$i=3$

$X_{32} \to X_{31} X_{41}$
$X_{32} \to \{B,C\}\{A\} = \{BA, CA\}$
$X_{32} = \{S,C\}$

$j=3 \quad 1 \le k \le 2$

$i=1$

$X_{13} \to X_{11} X_{22} \cup X_{12} X_{31}$
$X_{13} \to \{A\}\{B,S\} \cup \{\}\{B,C\} = \{AB, AS\}$
$X_{13} = \{B\}$

$i=2$

$X_{23} \to X_{21} X_{32} \cup X_{22} X_{41}$
$X_{23} \to \{A\}\{S,C\} \cup \{B,S\}\{A\} = \{AS, AC, BA, SA\}$
$X_{23} = \{S,C\}$

$j=4 \quad 1 \le k \le 3$

$i=1$

$X_{14} \to X_{11} X_{23} \cup X_{12} X_{32} \cup X_{13} X_{41}$
$X_{14} \to \{A\}\{S,C\} \cup \{\}\{S,C\} \cup \{B\}\{A\} = \{AS, AC, BA\}$
$X_{14} = \{S,C\}$

$S \in X_{14}$

Agenda

1. **Grammars and Languages**
   - Topic presentation
   - Exercises